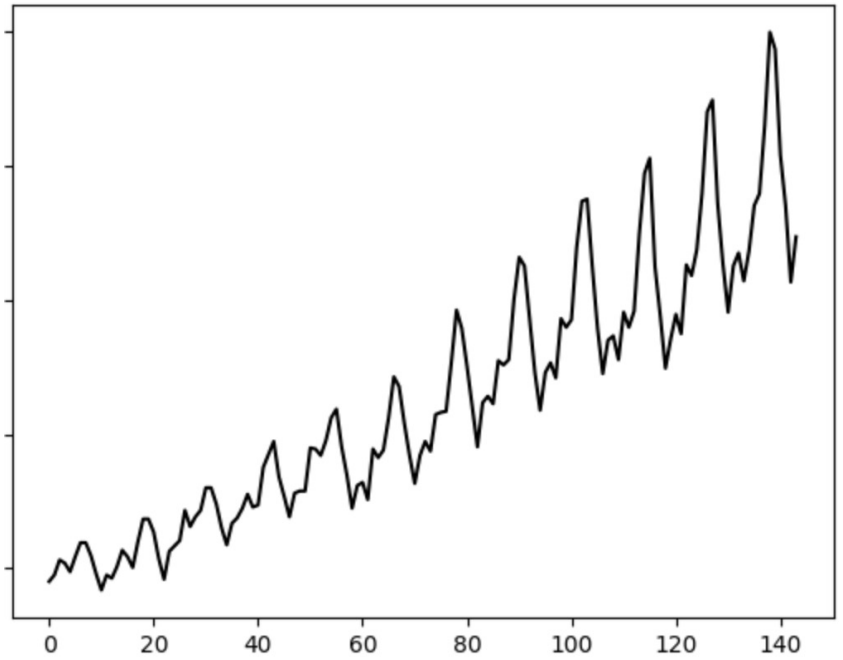
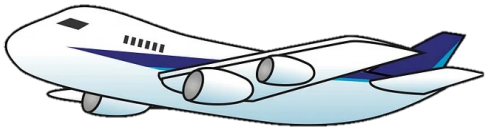
Sequence Modeling: Recurrent Neural Nets

Machine Learning on Sequential Data

- Example Sequential Data:

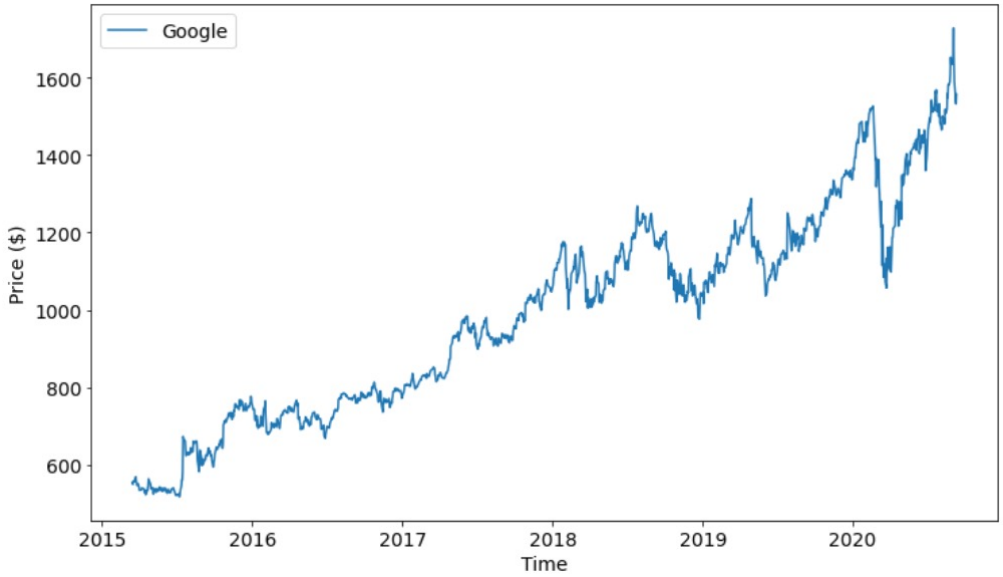




- Airline Passengers: Total number of monthly international passengers during 1949-1960.
- Goal: Predict number of passengers next month.

Machine Learning on Sequential Data

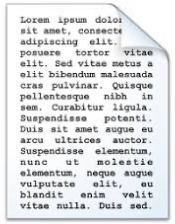
- Example Sequential Data:



- Stock Prices: Historic development of Google stock prices.
- Goal: Predict stock price tomorrow.

Machine Learning on Sequential Data

- Example Sequential Data:



"We shall execute the transaction under a legitimate…"

- Text: Variable length sentences
- Goal: Predict next word

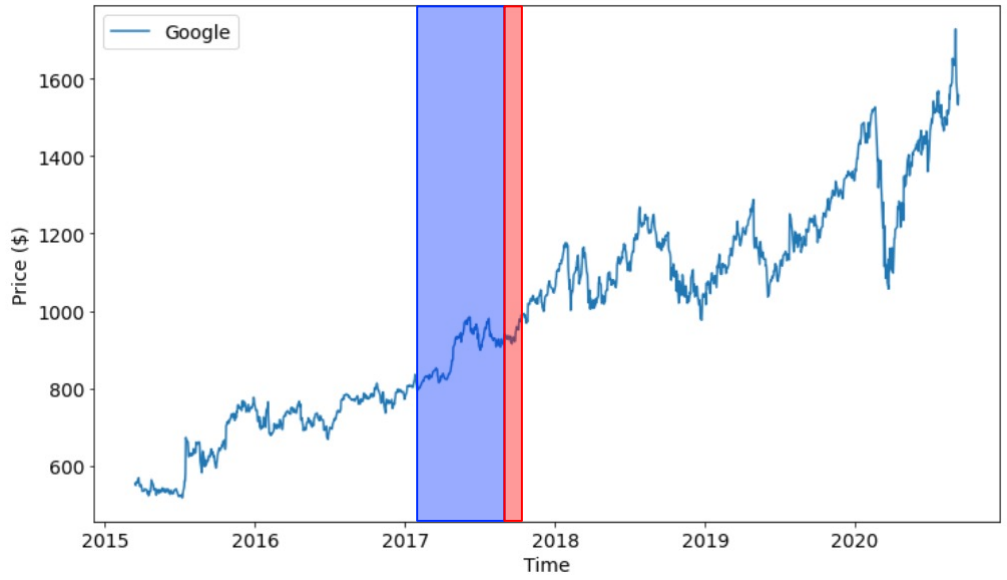
Machine Learning on Sequential Data

- How can we train to predict from sequential data?

- Modify existing models and algorithms?
  - Let us assume fixed length sequences
    - E.g. always 50 sequential data points/steps

Features

Label



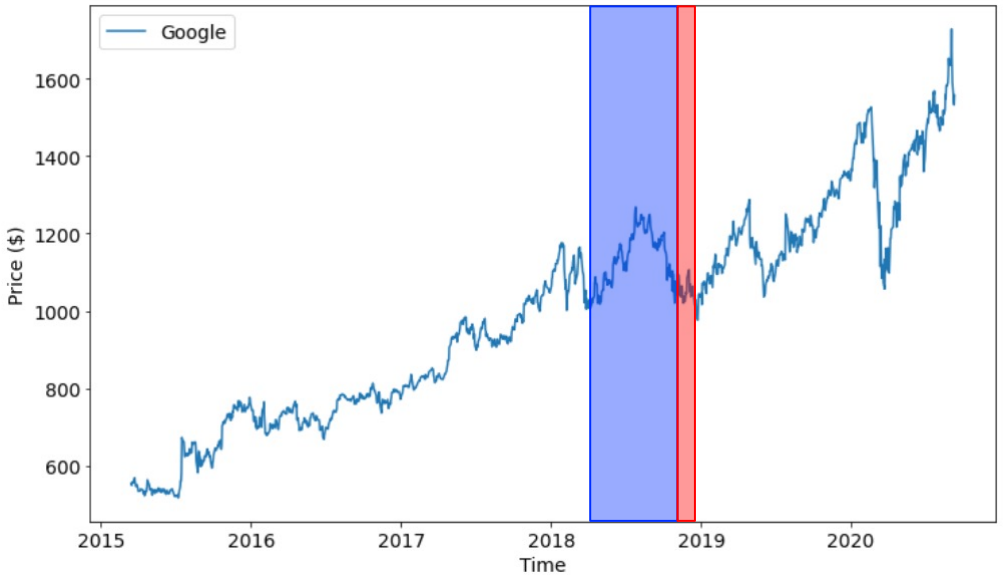
Machine Learning on Sequential Data

- How can we train to predict from sequential data?

- Modify existing models and algorithms?
  - Let us assume fixed length sequences
    - E.g. always 50 sequential data points/steps

Features

Label



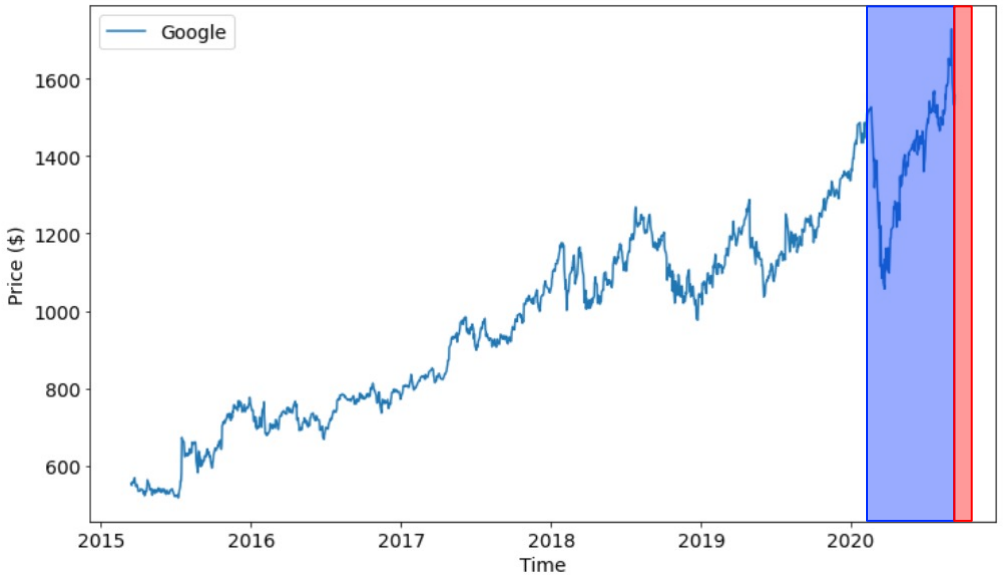
Machine Learning on Sequential Data

- How can we train to predict from sequential data?

- Modify existing models and algorithms?
  - Let us assume fixed length sequences
    - E.g. always 50 sequential data points/steps

Features

Label



Machine Learning on Sequential Data

- How can we train to predict from sequential data?

- Modify existing models and algorithms?
  - Let us assume fixed length sequences
    - E.g. always 50 sequential data points/ste

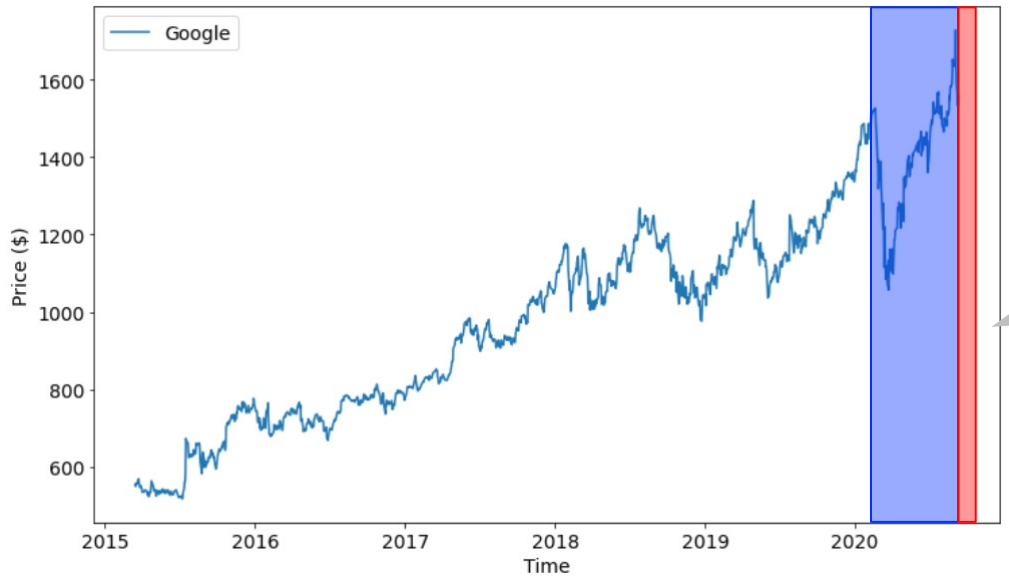
Every time step has features:

stock price Google, inflation,

stock price Apple

Features

Label



Label is stock price tomorrow

Sequential Data

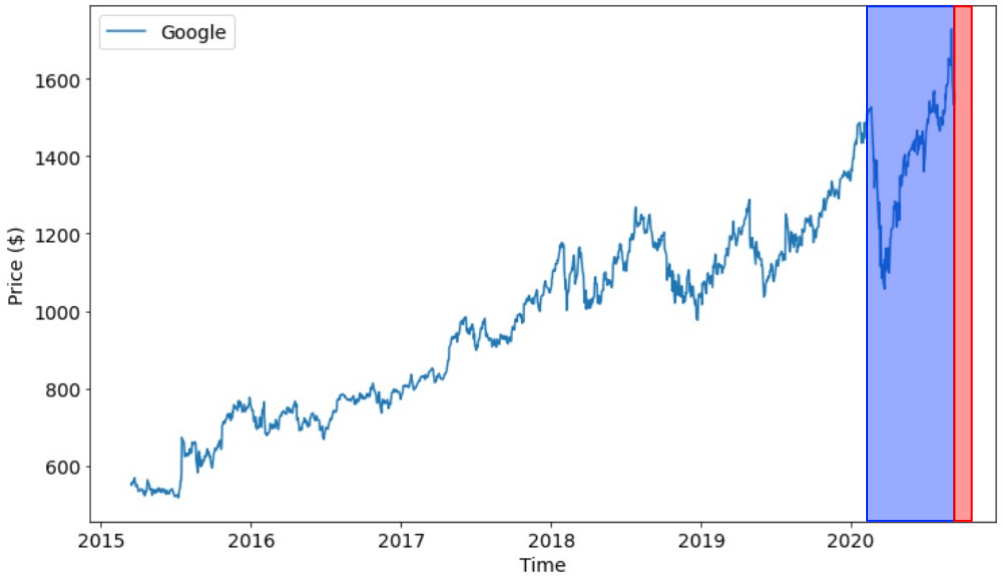
- Modifying existing algorithms?

Step 1 features | Step 3 features | Step 50 features

Feature Vector (0,… …,5,7, … … …,0,1,0)

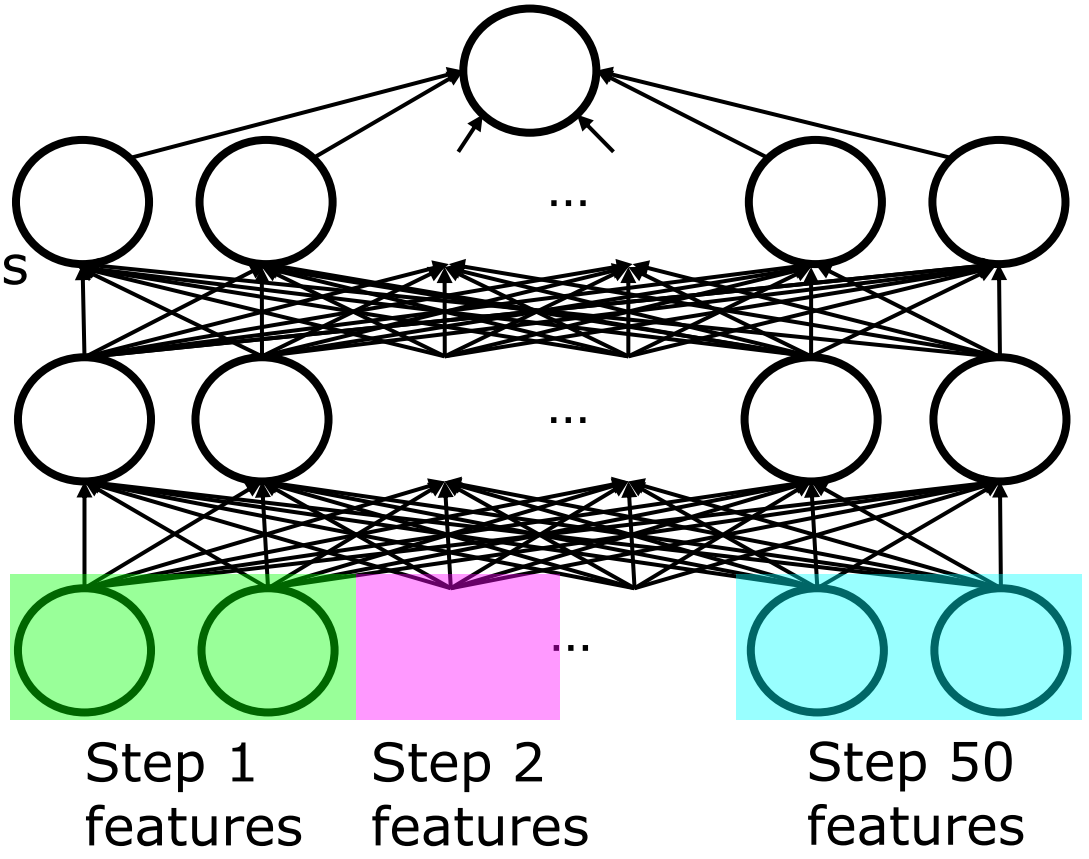
Features

Label



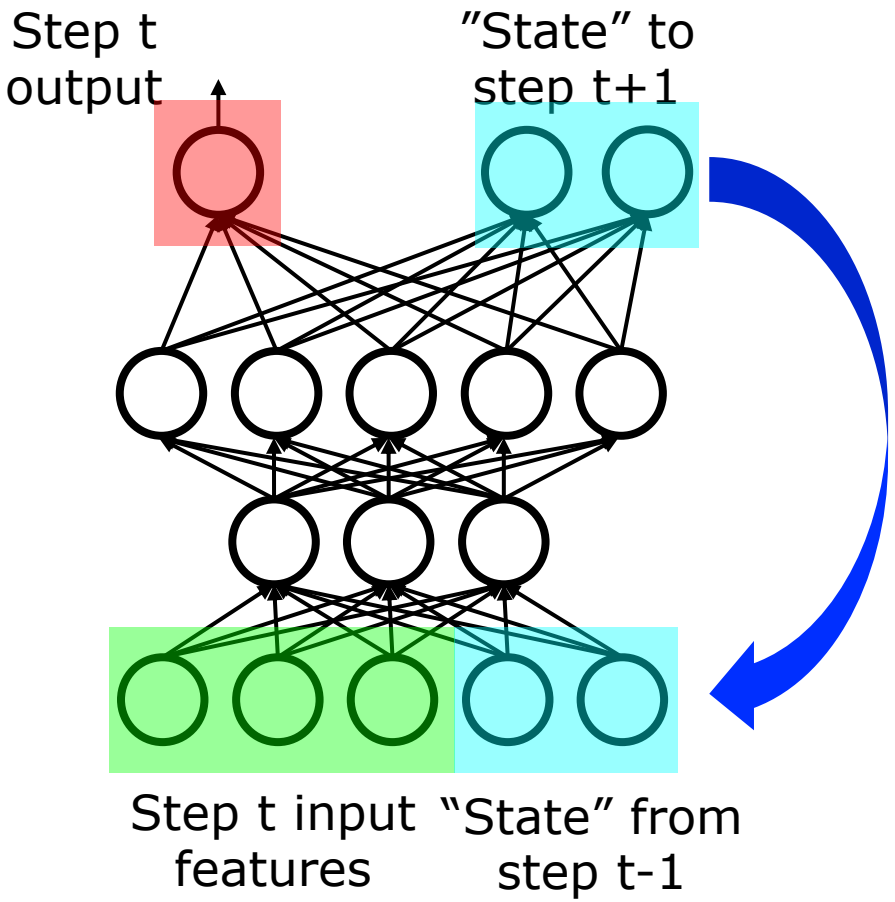
Sequential Data

- Modifying existing algorithms?
  - Doable
- Downsides:
  - Many parameters/weights
  - Probably steps should be treated similarly, so need to re-learn same functionality in many ”places”
  - Cannot handle variable length sequences



Recurrent Neural Nets

- Variant of Neural Nets for sequential data



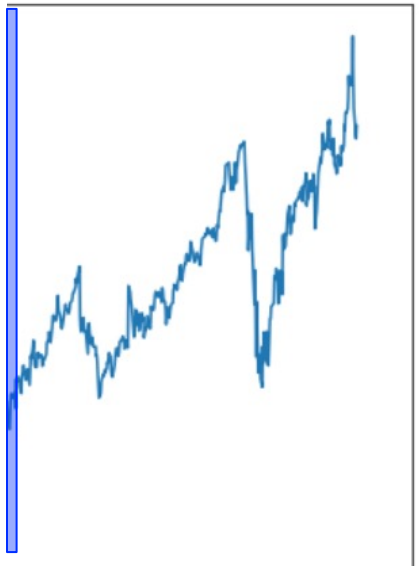
High-level
structure

Recurrent Neural Nets

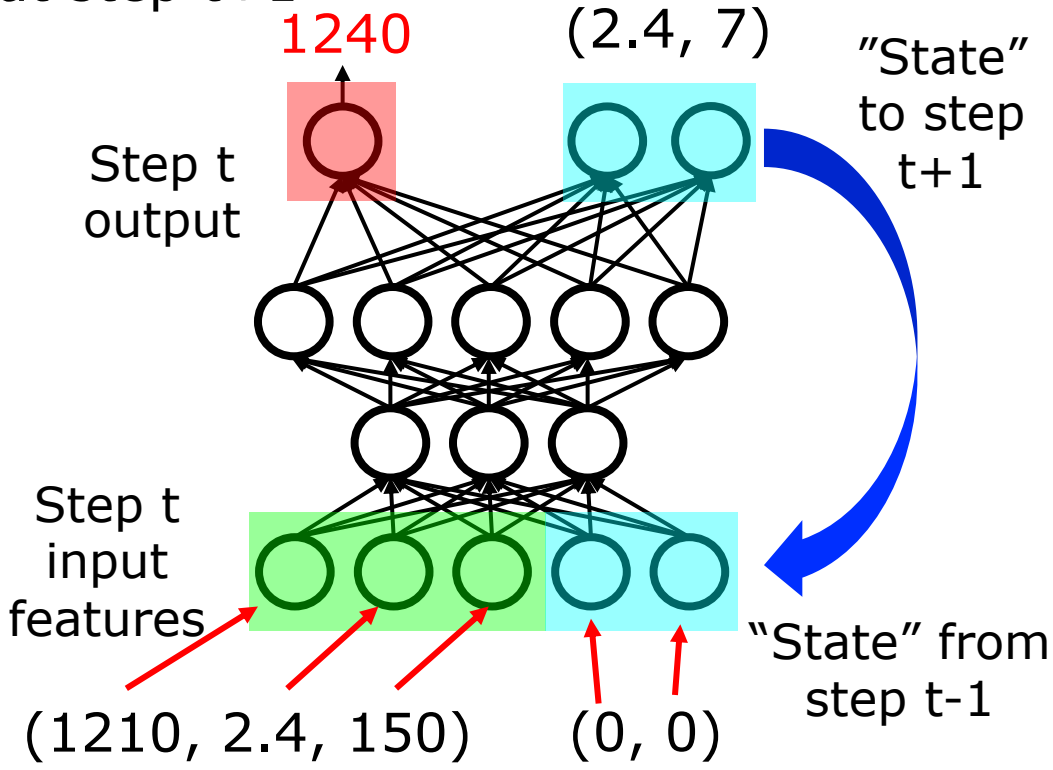
- High-Level Idea

  Features at step t: stock price Google, inflation, stock price Apple

  Label: stock price Google at step t+1



Step 1

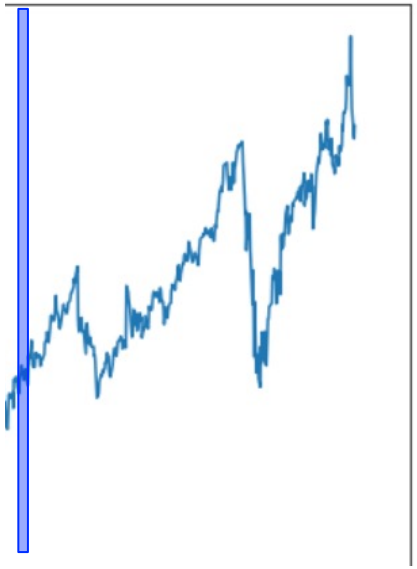


Recurrent Neural Nets

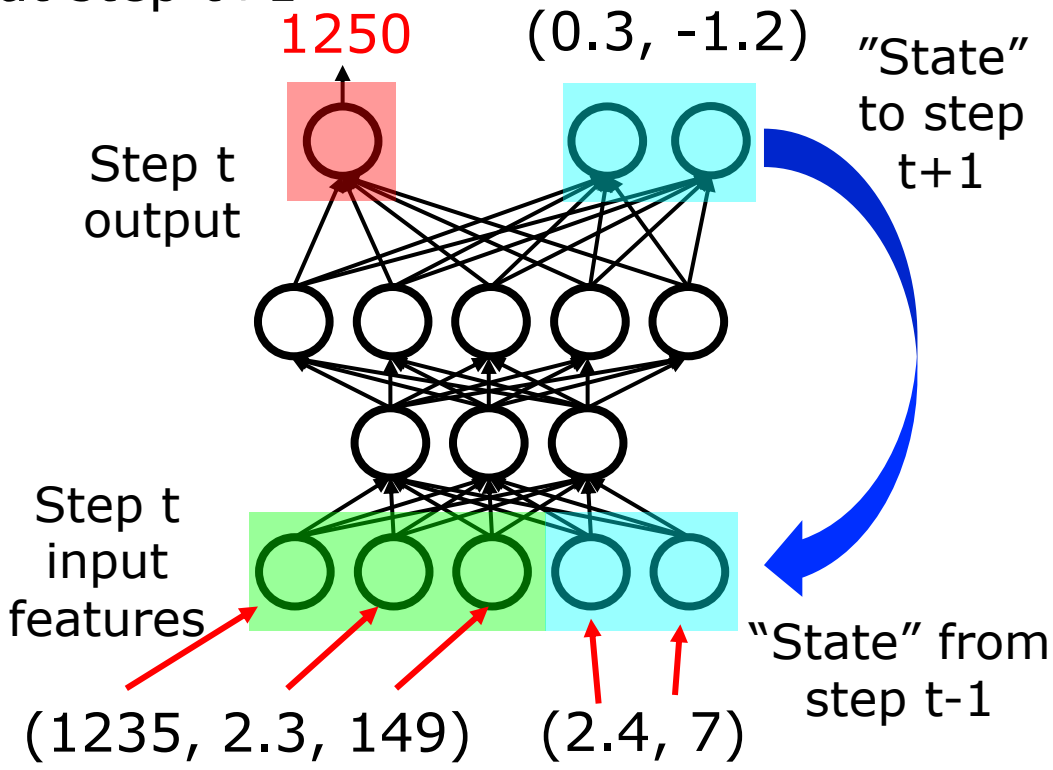
- High-Level Idea

  Features at step t: stock price Google, inflation, stock price Apple

  Label: stock price Google at step t+1



Step 2

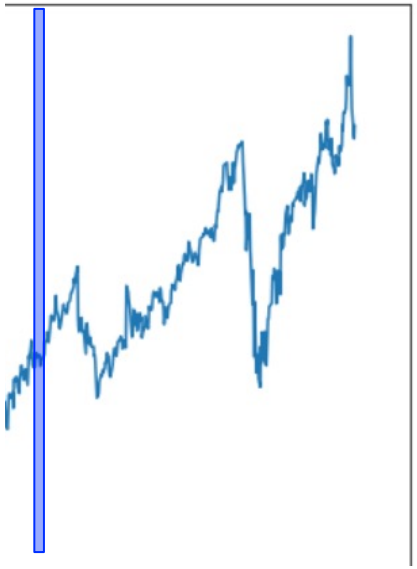


Recurrent Neural Nets

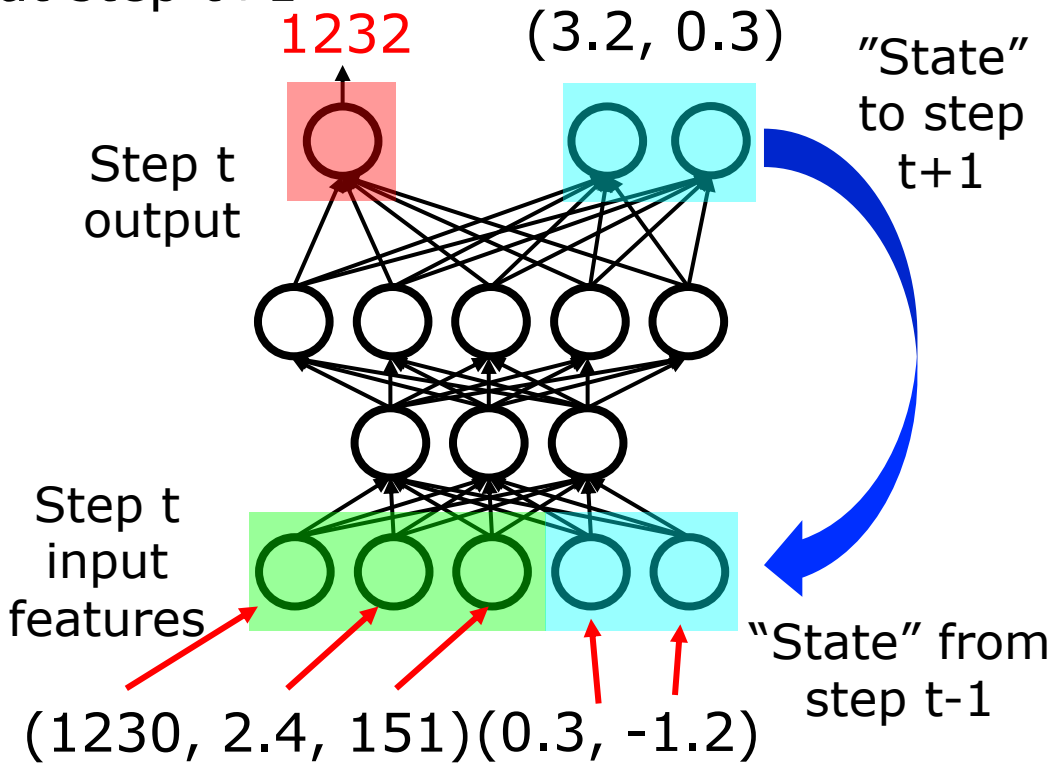
- High-Level Idea

  Features at step t: stock price Google, inflation, stock price Apple

  Label: stock price Google at step t+1



Step 3

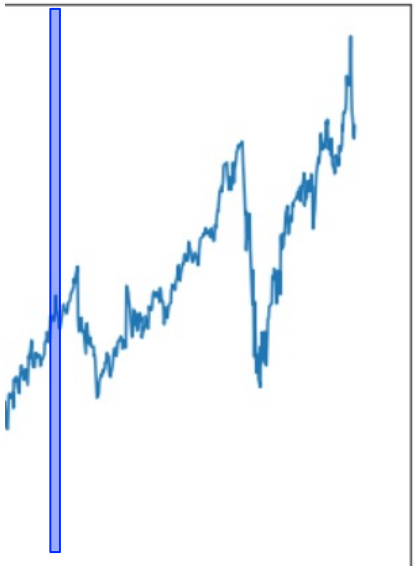


Recurrent Neural Nets

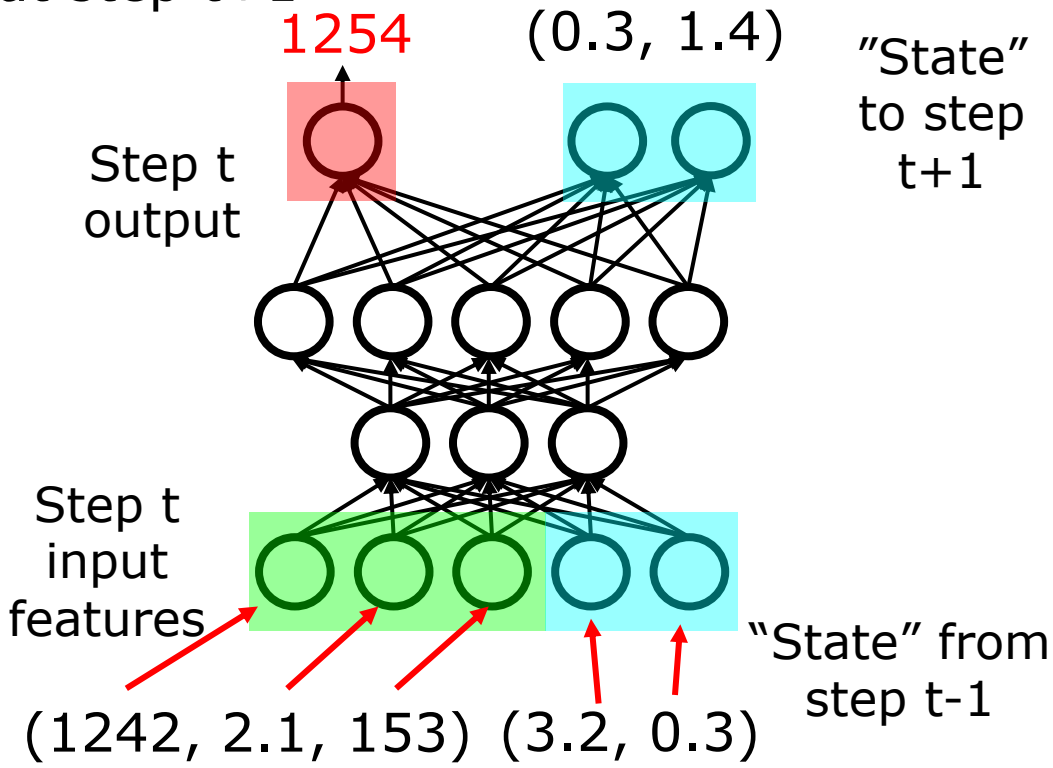
- High-Level Idea

  Features at step t: stock price Google, inflation, stock price Apple

  Label: stock price Google at step t+1



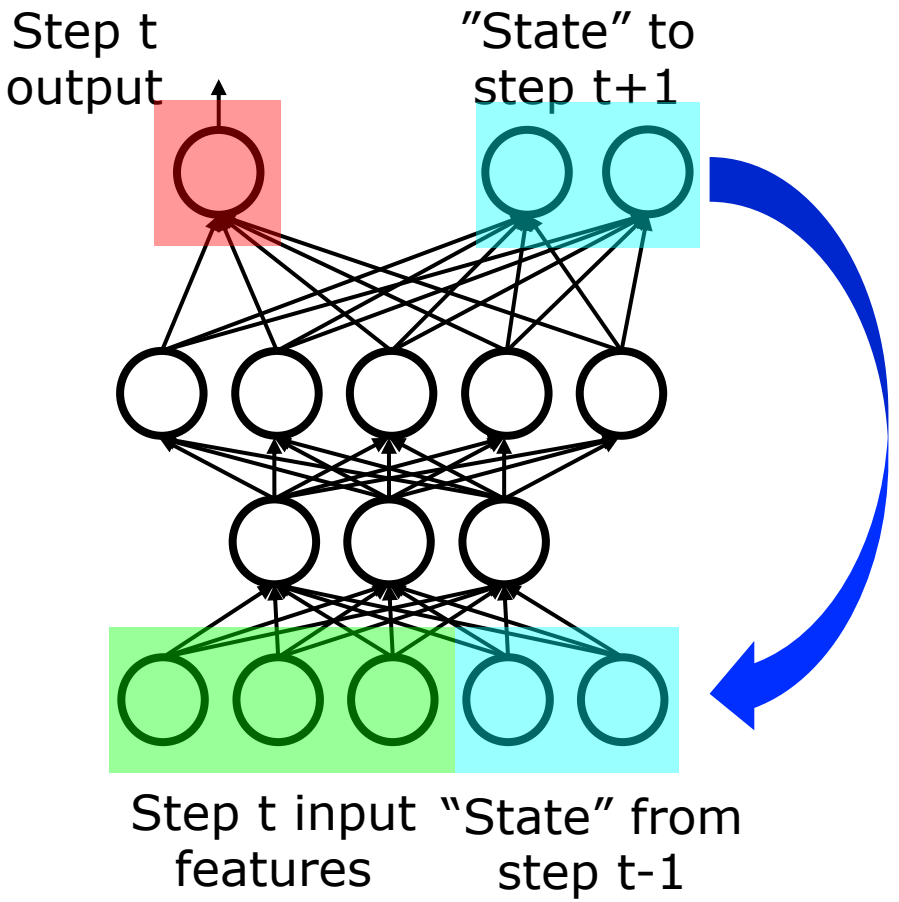
Step 4



Recurrent Neural Nets

- Variant of Neural Nets for sequential data

High-level structure



Benefits:

Sharing of parameters across steps

Arbitrary length sequences

Fewer weights

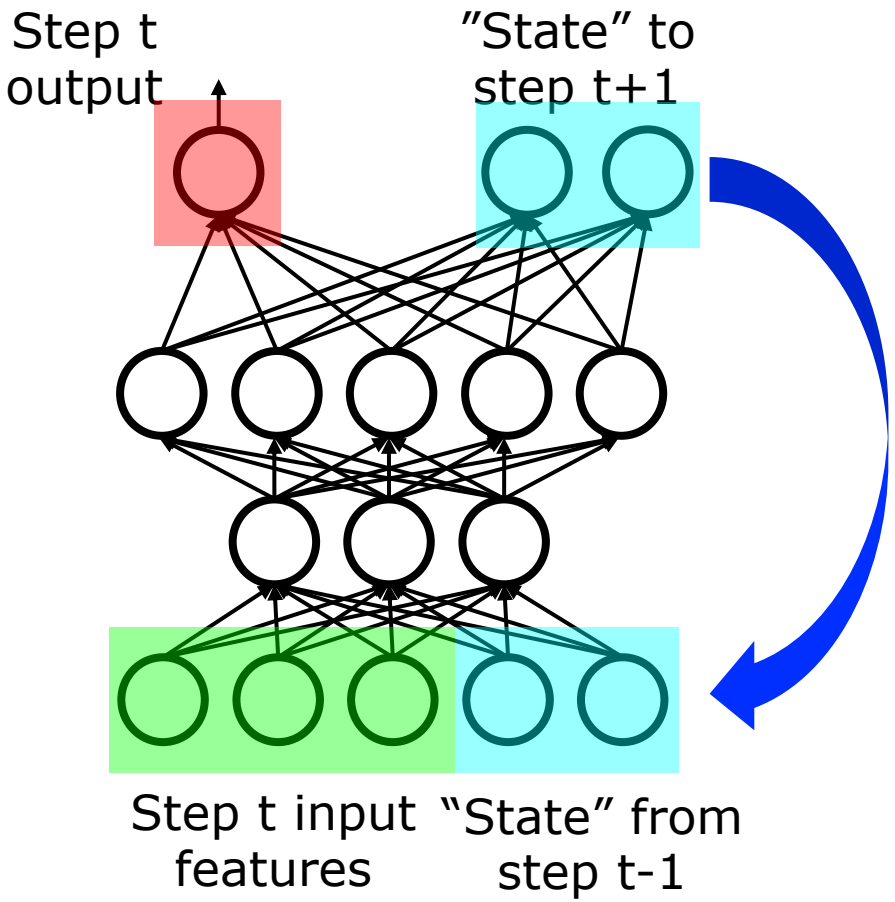
Forwarding of state to remember important info on sequence

Recurrent Neural Nets

- Variant of Neural Nets for sequential data

Next:

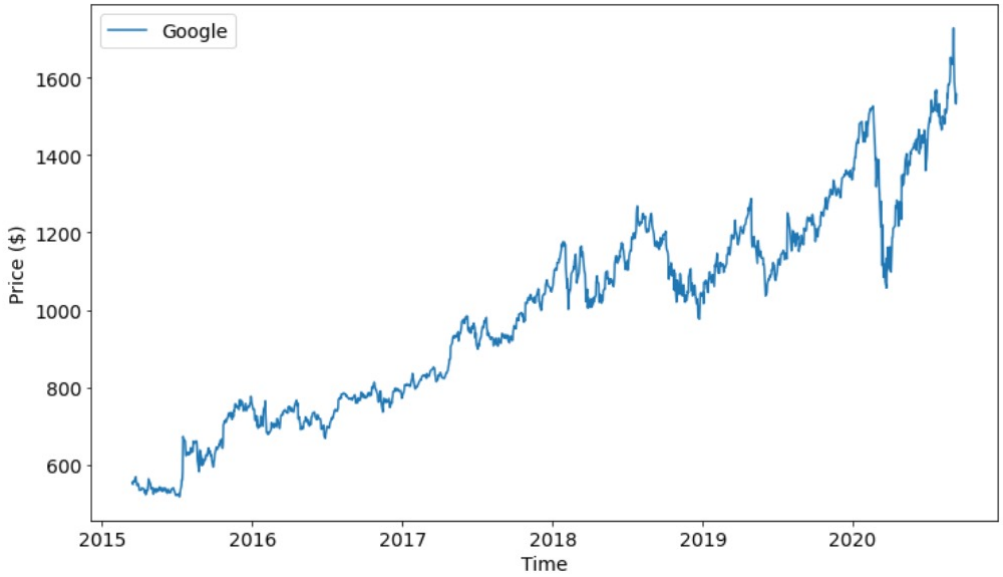
How to train them



Training Recurrent Neural Nets

Machine Learning on Sequential Data

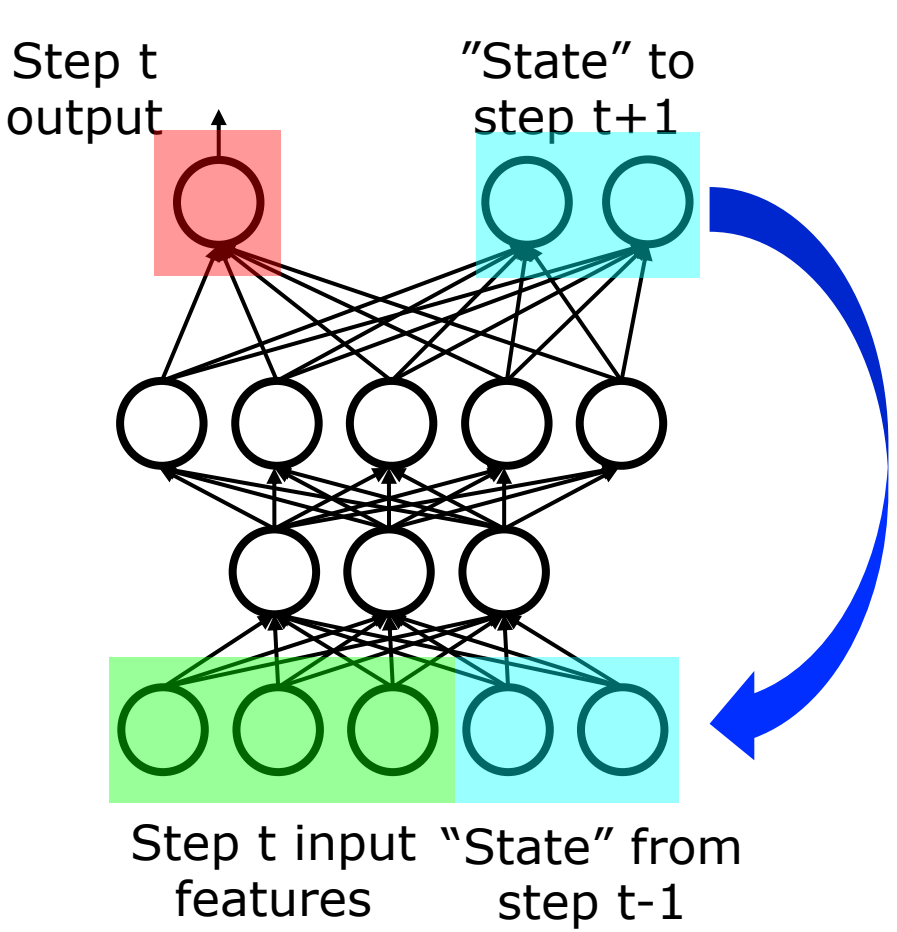
- Example Sequential Data:



- Stock Prices: Historic development of Google stock prices.
- Goal: Predict stock price tomorrow.

Recurrent Neural Nets

- Variant of Neural Nets for sequential data

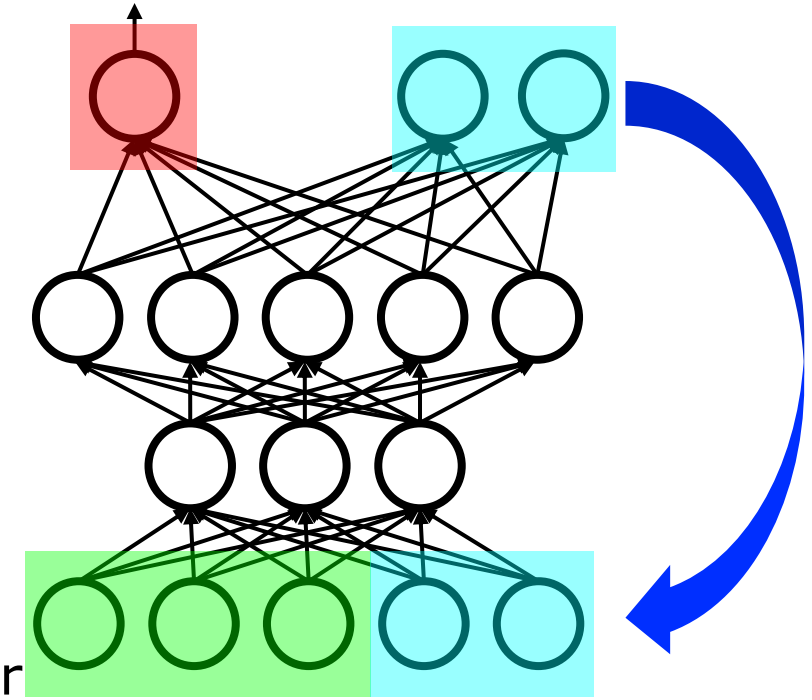


Training Recurrent Neural Nets

- Training data format:
  - Features: $(x^1, x^2, \ldots, x^t, \ldots)$
  - Labels: $(y^1, y^2, \ldots, y^t, \ldots)$

- Feature vector x^t in R^d for step t
  - Example:
    - (Google stock price step t, inflation, Apple stock price)

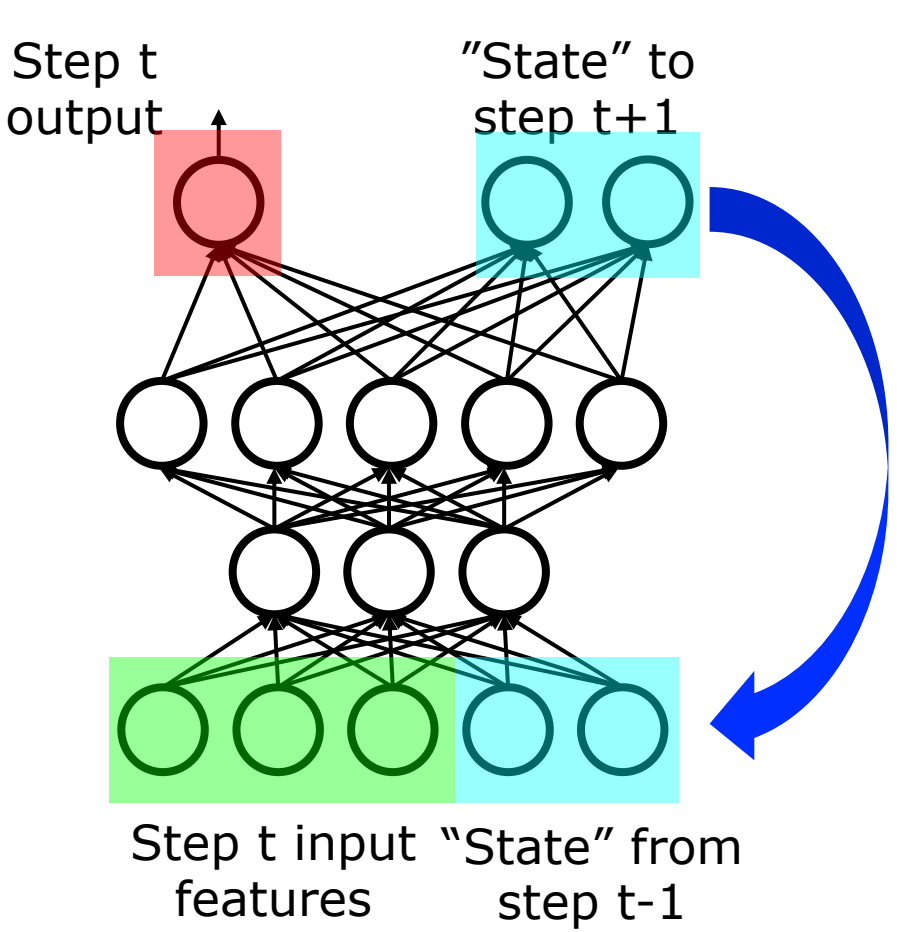
- Target label y^t for step t
  - E.g. -1/1 or one-hot-encoded vector
  - Example:
    - Google stock price step t+1



What we would like to be able to predict using model

Training Recurrent Neural Nets

- Let us consider simplest recurrent architechture



Training Recurrent Neural Nets

- Let us consider simplest recurrent architechture

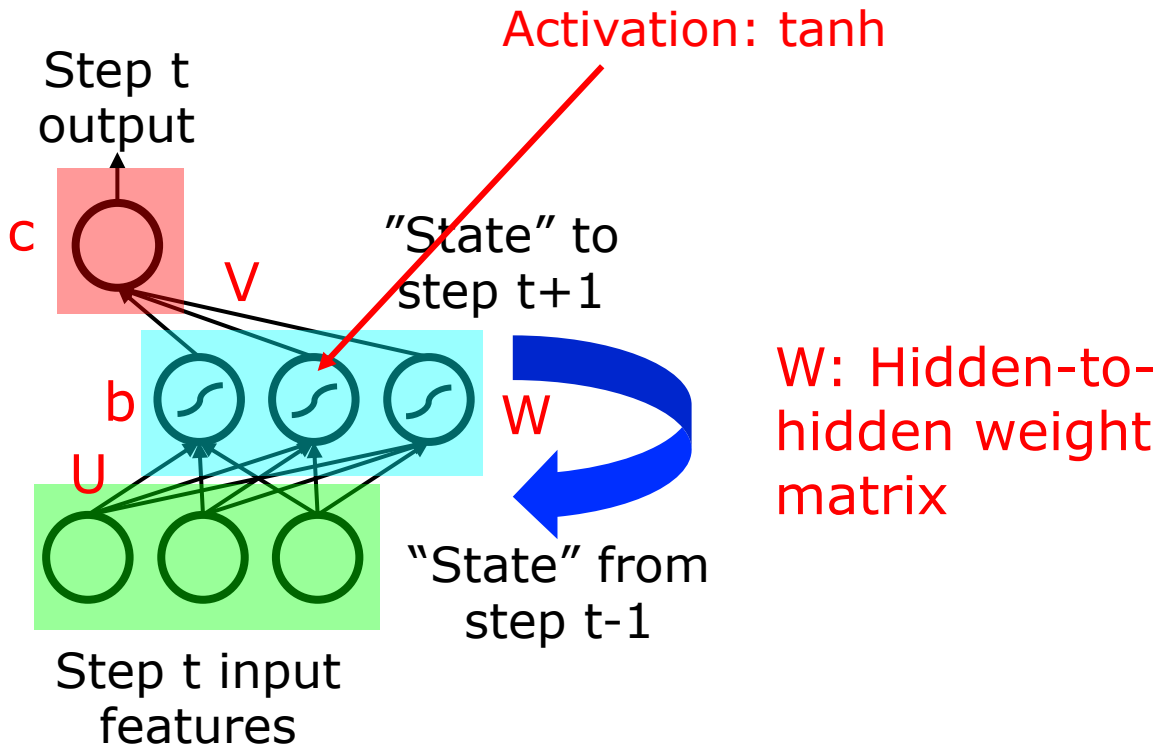
c: Output biases

V: Hidden-to-output weight matrix

One hidden layer

U: Input-to-hidden weight matrix

b: Hidden biases

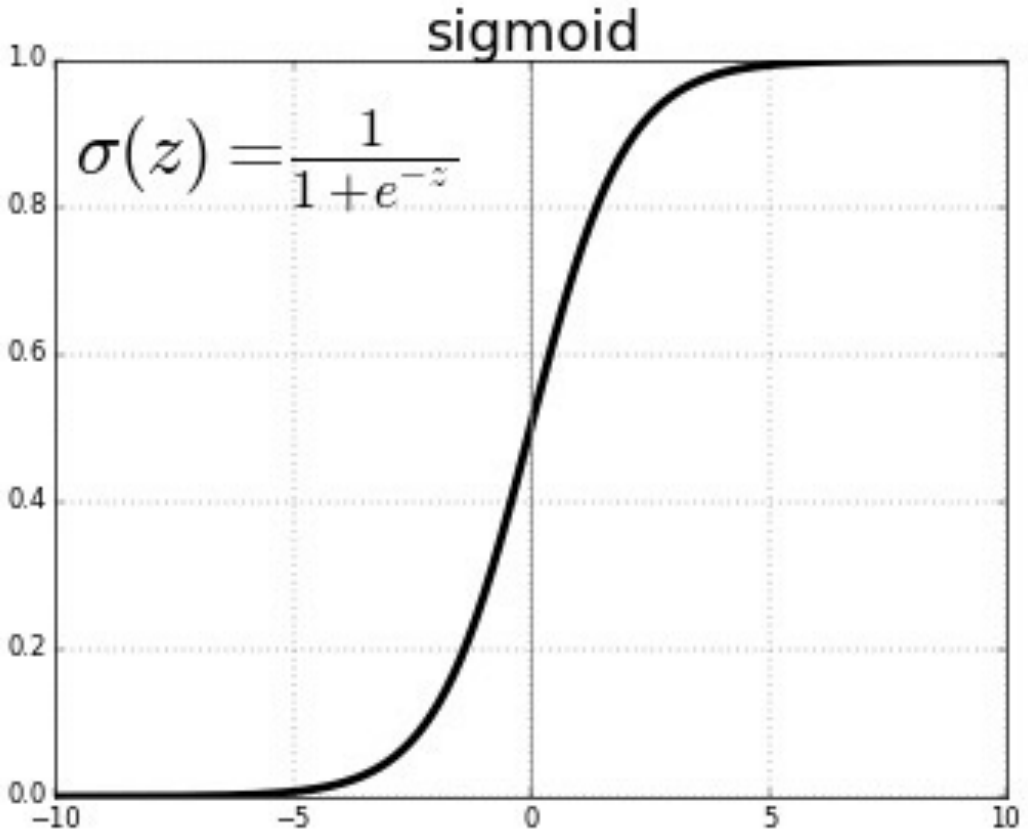


W: Hidden-to-hidden weight matrix

Sigmoid

- Logistic Sigmoid

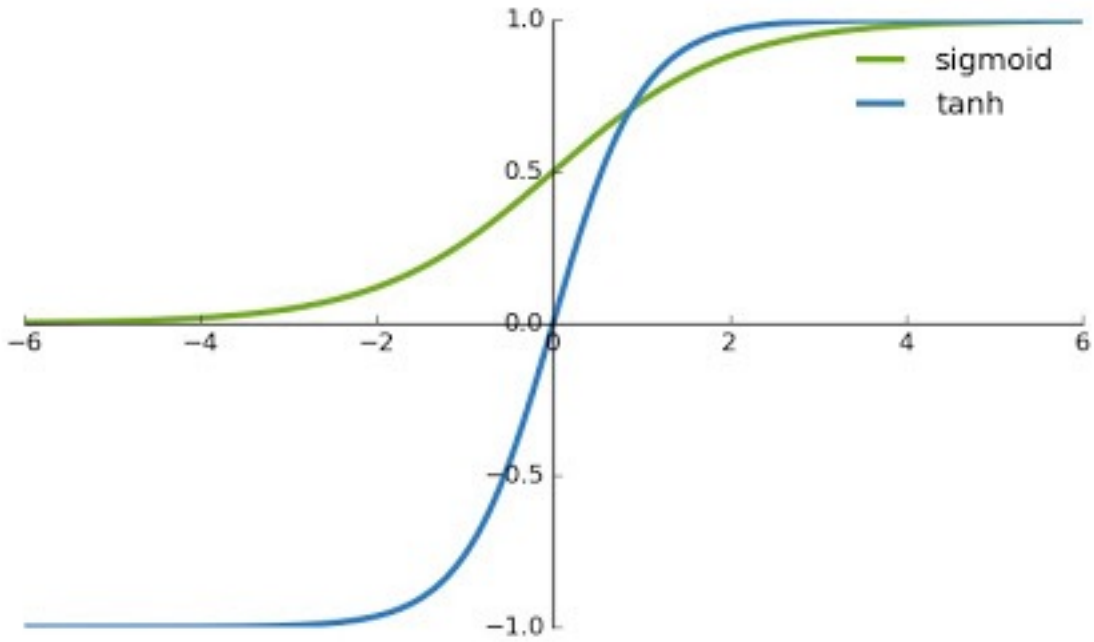
- Sigmoid $\sigma(z)$
  - ”Fires” as z becomes big enough.



Hyperbolic Tangent

- Hyperbolic Tangent

- $\tanh(z)$
  - $\tanh(z) = 2\sigma(2z) - 1$

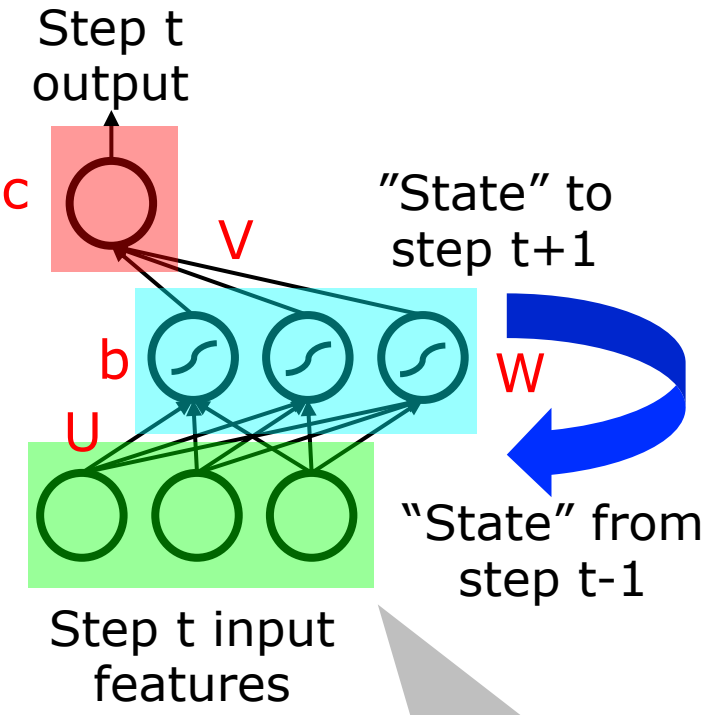


- We will return to why tanh is a good choice for Recurrent Neural Nets!

Training Recurrent Neural Nets

- Let us consider simplest recurrent architechture
  - d input features, h hidden neurons, o output neurons

c: 1 x o
V: h x o
U: d x h
W: h x h
b: 1 x h



Step 1:

$h^0=(0,0,0)$

$h^1=\tanh(x^1U+h^0W+b)$

$o^1=h^1V+c$

Step 2:

$h^2=\tanh(x^2U+h^1W+b)$

$o^2=h^2V+c$

Step 3:

$h^3=\tanh(x^3U+h^2W+b)$

$o^3=h^3V+c$

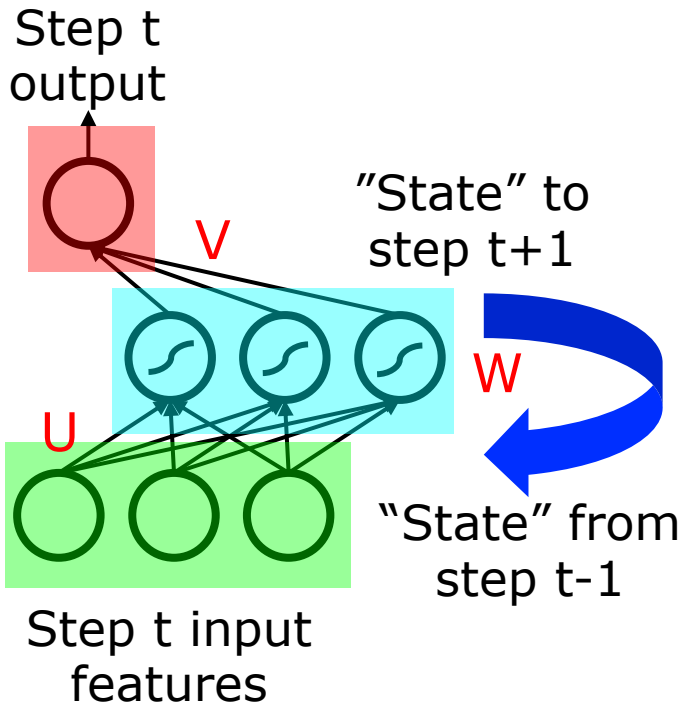
- Features: $(x^1,x^2,\ldots,x^t,\ldots)$

For simpler notation, let us also drop biases in example

Training Recurrent Neural Nets

- Let us consider simplest recurrent architechture
  - d input features, h hidden neurons, o output neurons

V: h x o
U: d x h
W: h x h



Step 1:

$h^0=(0,0,0)$

$h^1=\tanh(x^1U+h^0W)$

$o^1=h^1V$

Step 2:

$h^2=\tanh(x^2U+h^1W)$

$o^2=h^2V$

Step 3:

$h^3=\tanh(x^3U+h^2W)$

$o^3=h^3V$

- Features: $(x^1,x^2,\ldots,x^t,\ldots)$

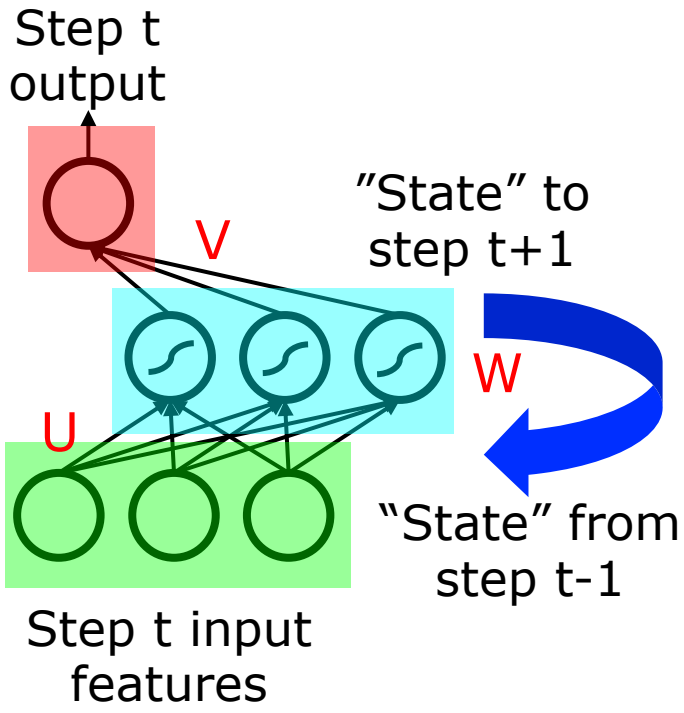
Training Recurrent Neural Nets

- Let us consider simplest recurrent architechture
  - d input features, h hidden neurons, o output neurons

V: h x o

U: d x h

W: h x h



Step t:

$h^t = \tanh(x^t U + h^{t-1} W)$

$o^t = h^t V$

- Features: $(x^1, x^2, \ldots, x^t, \ldots)$

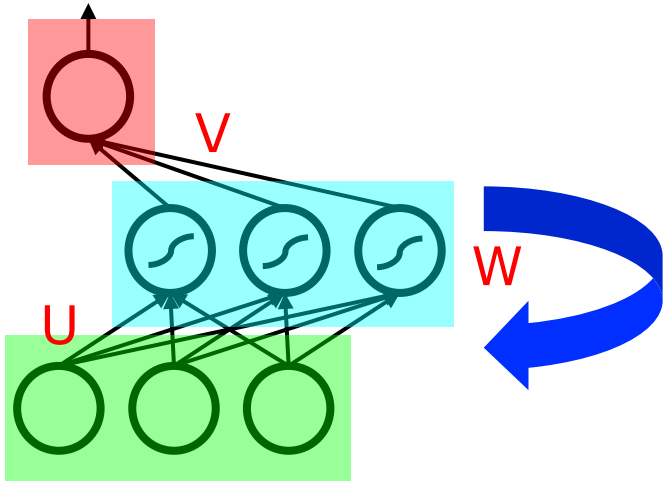
Training Recurrent Neural Nets

- Loss for training
  - Loss function L(y',y), e.g. Least-Squares/Cross-Entropy etc.

How can we minimize this loss?

- On n training seqs.
  - $s_1,\ldots,s_n$

  Loss: $\frac{1}{n}\sum_i L(s_i)$



Step t:

$h^t = \tanh(x^tU + h^{t-1}W)$

$o^t = h^tV$

- Sequence s:
  - Labels: $(y^1,y^2,\ldots,y^t,\ldots)$
  - Features: $(x^1,x^2,\ldots,x^t,\ldots)$

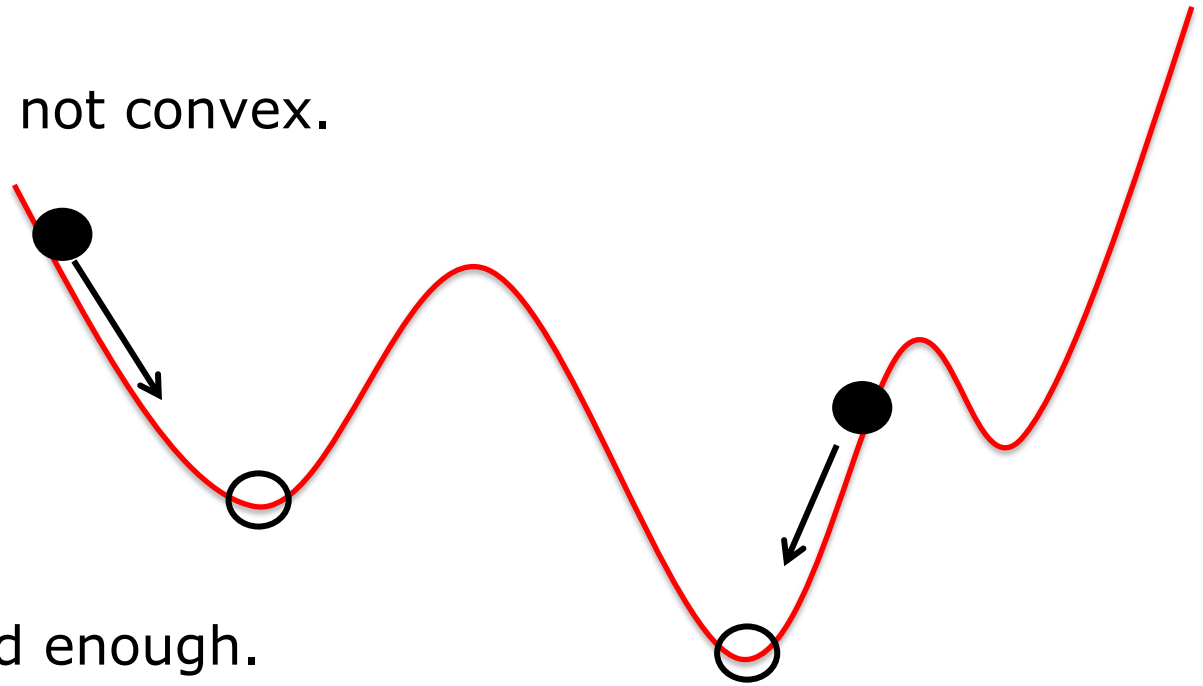
Loss on sequence s of length T:

$$L(s) = \sum_{t=1}^{T} L(o^t, y^t)$$

Training Recurrent Neural Nets

- (Mini-Batch) Stochastic Gradient Descent
  - Algorithm only needs gradients.
    - Gradient of loss function wrt. weights.

- Concern:
  - Loss of neural net not convex.



- Don't care:
  - Local minima good enough.

Stochastic Gradient Descent

- Any point-wise loss:
  - $\mathrm{E}_{\mathrm{in}}(\mathrm{U}, V, W) = \frac{1}{n}\sum_i \sum_{t=1}^{T} L(o^t(x_i), y_i^t)$
- Mini-batch SGD:
  - Initialize U,V,W (e.g. random N(0,1) distributed entries).
  - While more time:
    - Permute $(x_1,y_1),\ldots,(x_n,y_n)$ randomly.
    - For i=1,...,n/B:
      - Let $(x_1',y_1'),\ldots,(x_B',y_B')$ be next B data items.
      - $g = \frac{1}{B}\sum_{i=1}^{B} \sum_{t=1}^{T} \nabla_{U,V,W} L(o^t(x_i), y_i^t)$
      - U,V,W = U,V,W - ηg
  - Return w.

$o^t(x_i)$ output of recurrent neural net in step t on sequence $s_i=(x_i,y_i)$ with weights U,V,W

We need gradients

Backpropagation

- Backprop:
  - Efficient algorithm for computing gradients of computations.

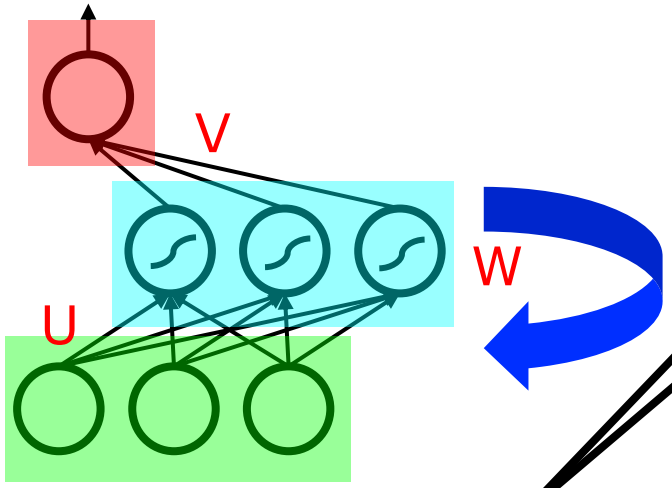
- Computation Graph:
  - We need to convert a recurrent neural net to a (vectorized) computation graph.

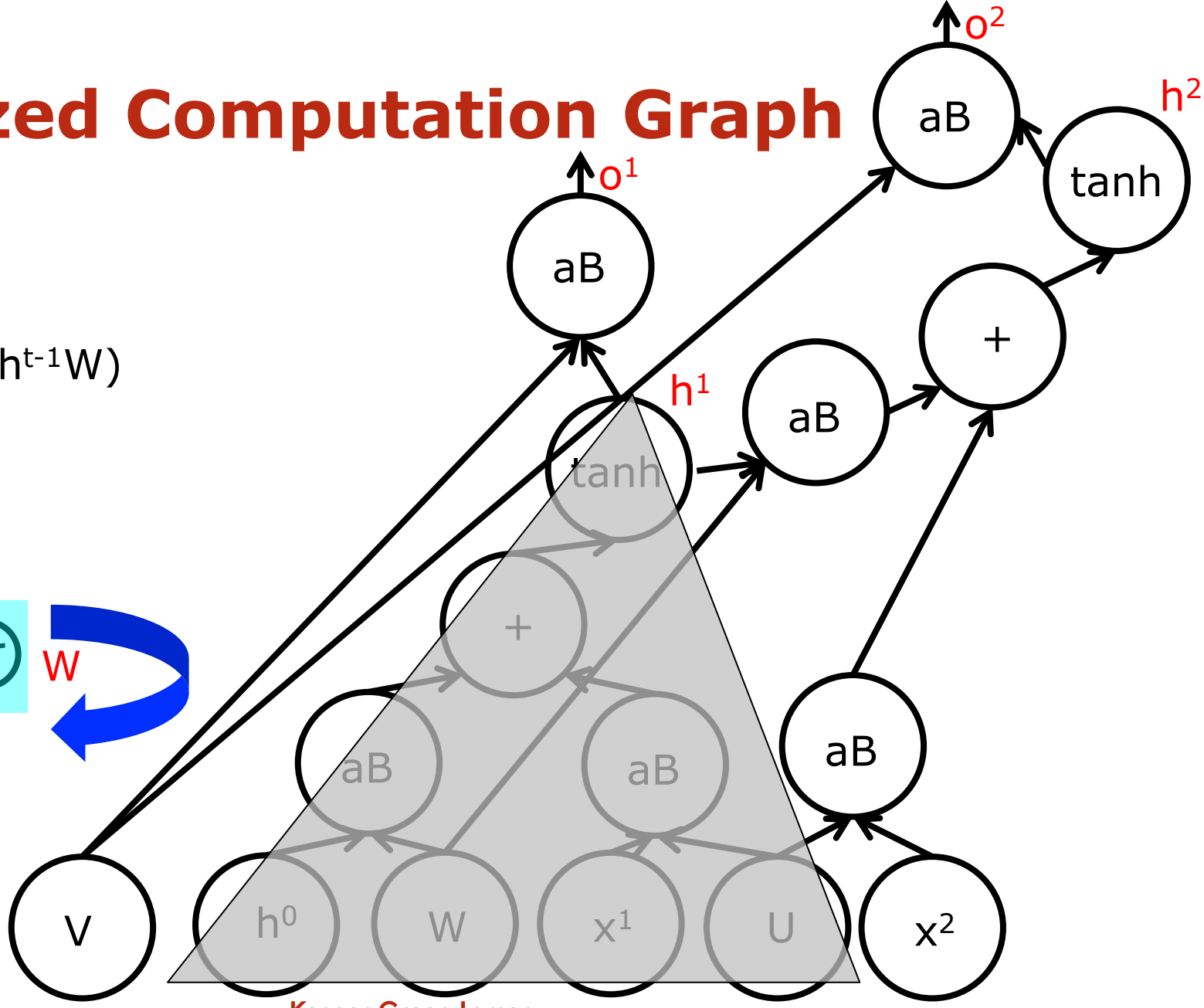
Vectorized Computation Graph

Step t:

$h^t = \tanh(x^tU + h^{t-1}W)$

$o^t = h^tV$



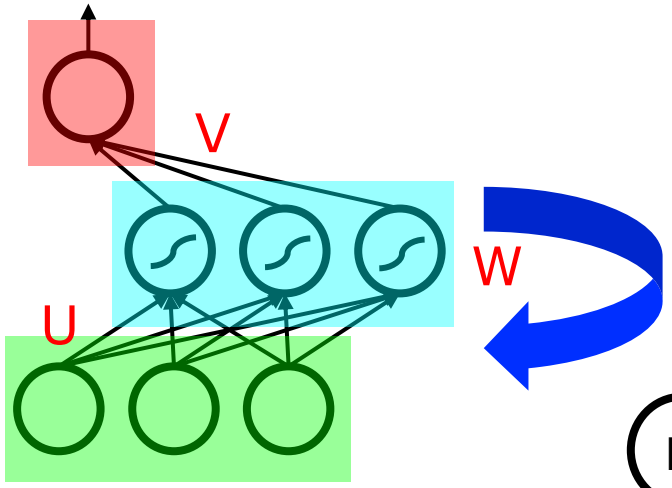


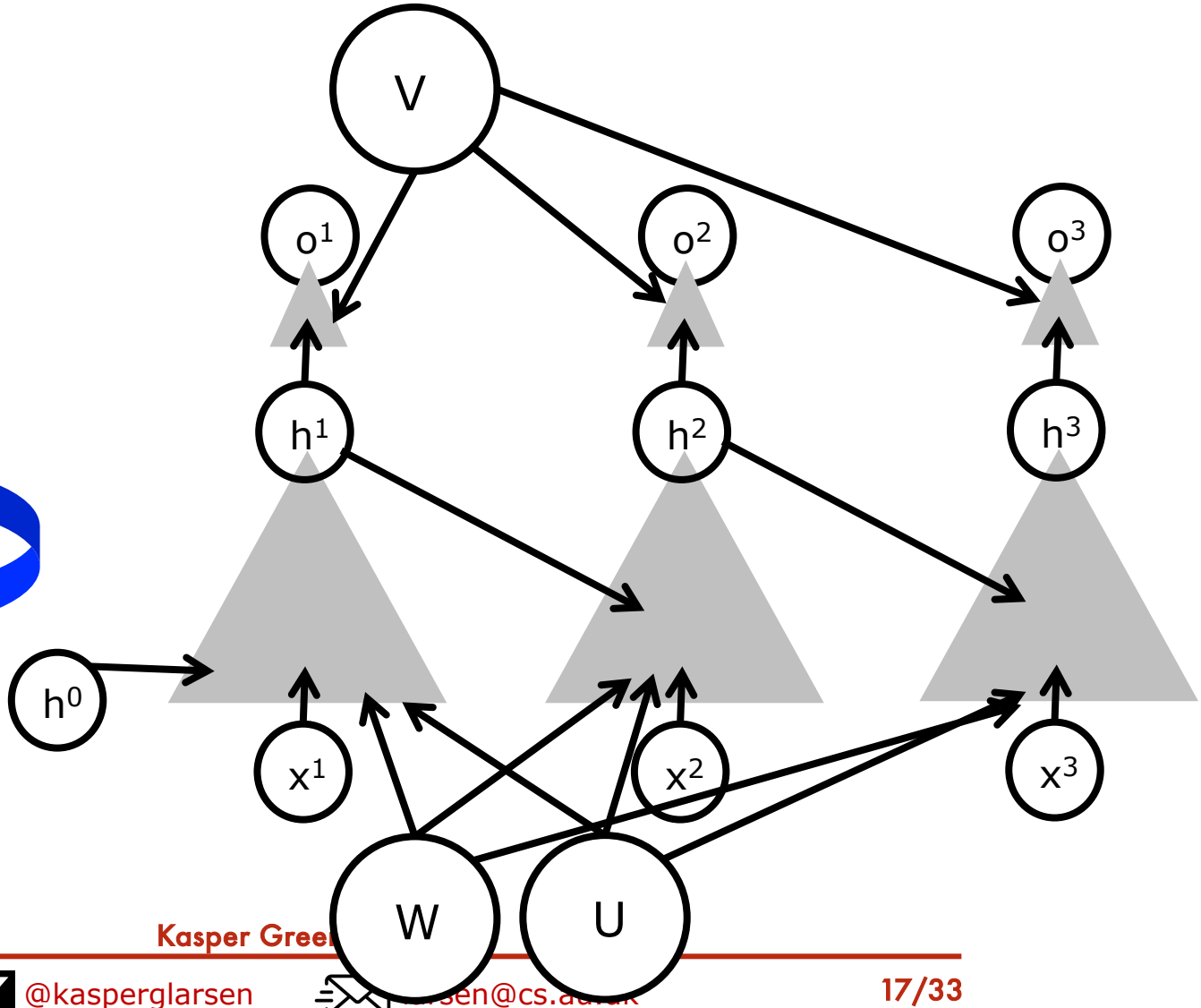
Vectorized Computation Graph

Step t:

$h^t = \tanh(x^t U + h^{t-1} W)$

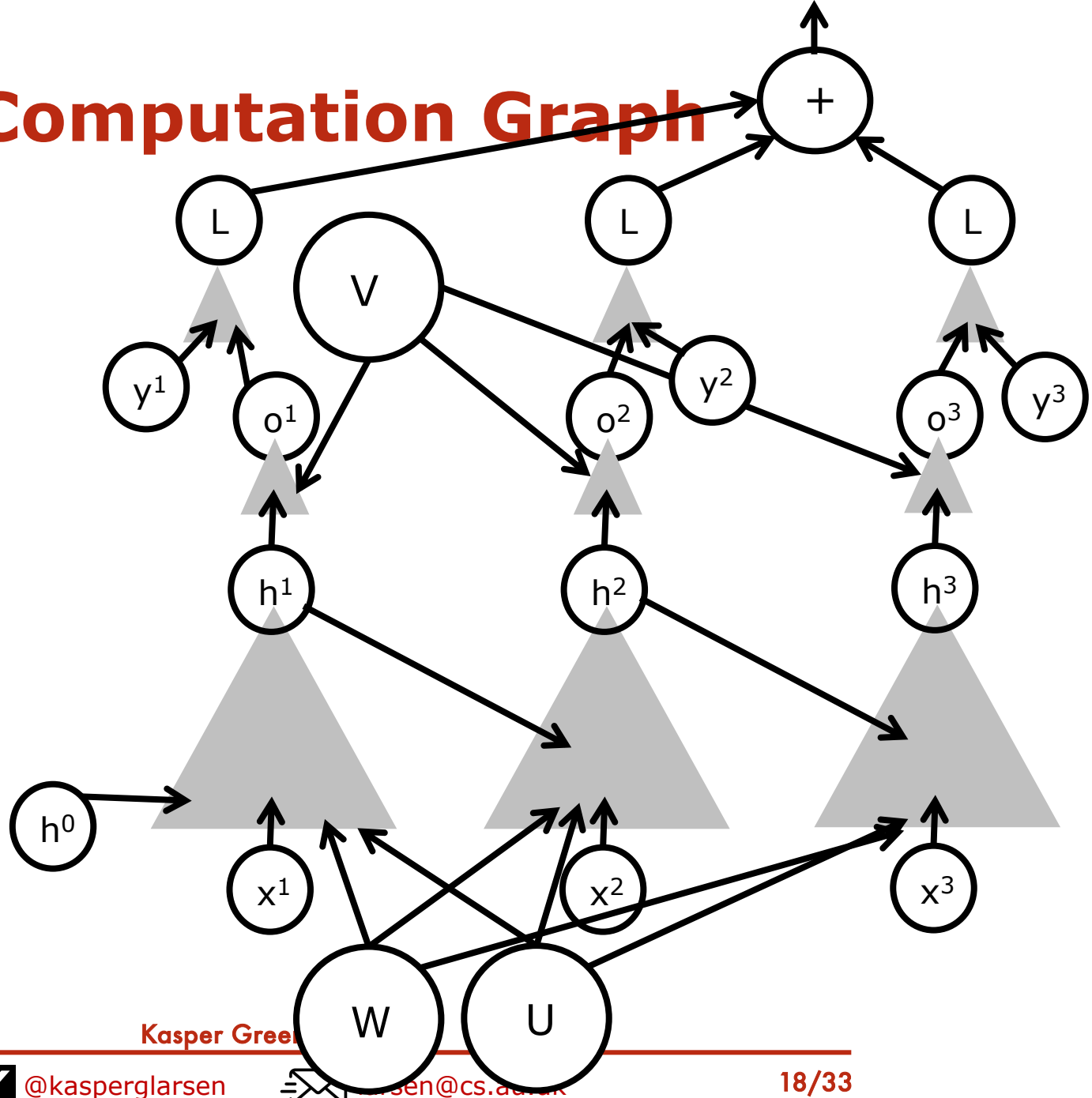
$o^t = h^t V$





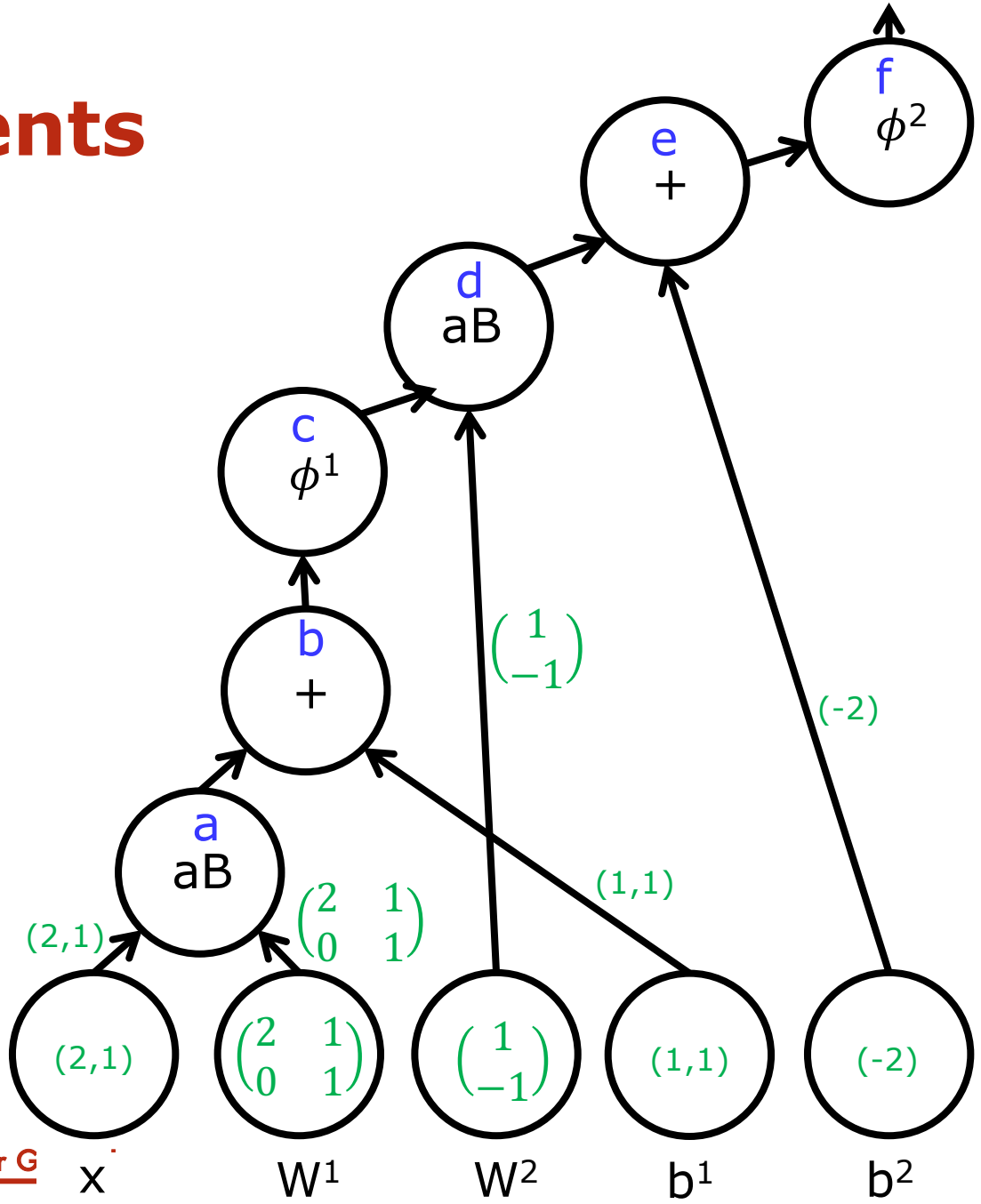
Vectorized Computation Graph

$$L(s) = \sum_{t=1}^{T} L(o^t, y^t)$$



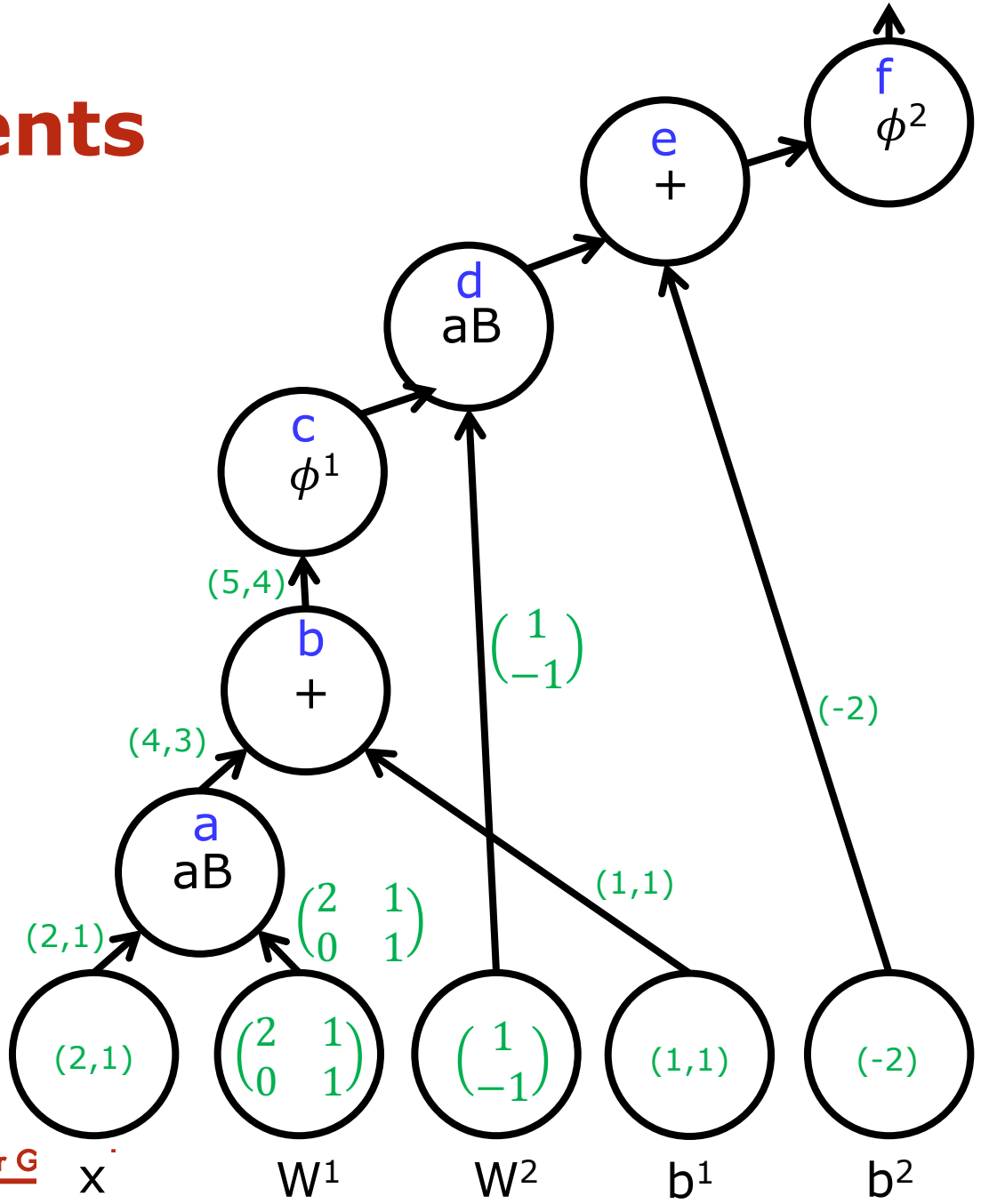
Computing Gradients

- We saw earlier:
  - Name nodes.
  - Forward pass.



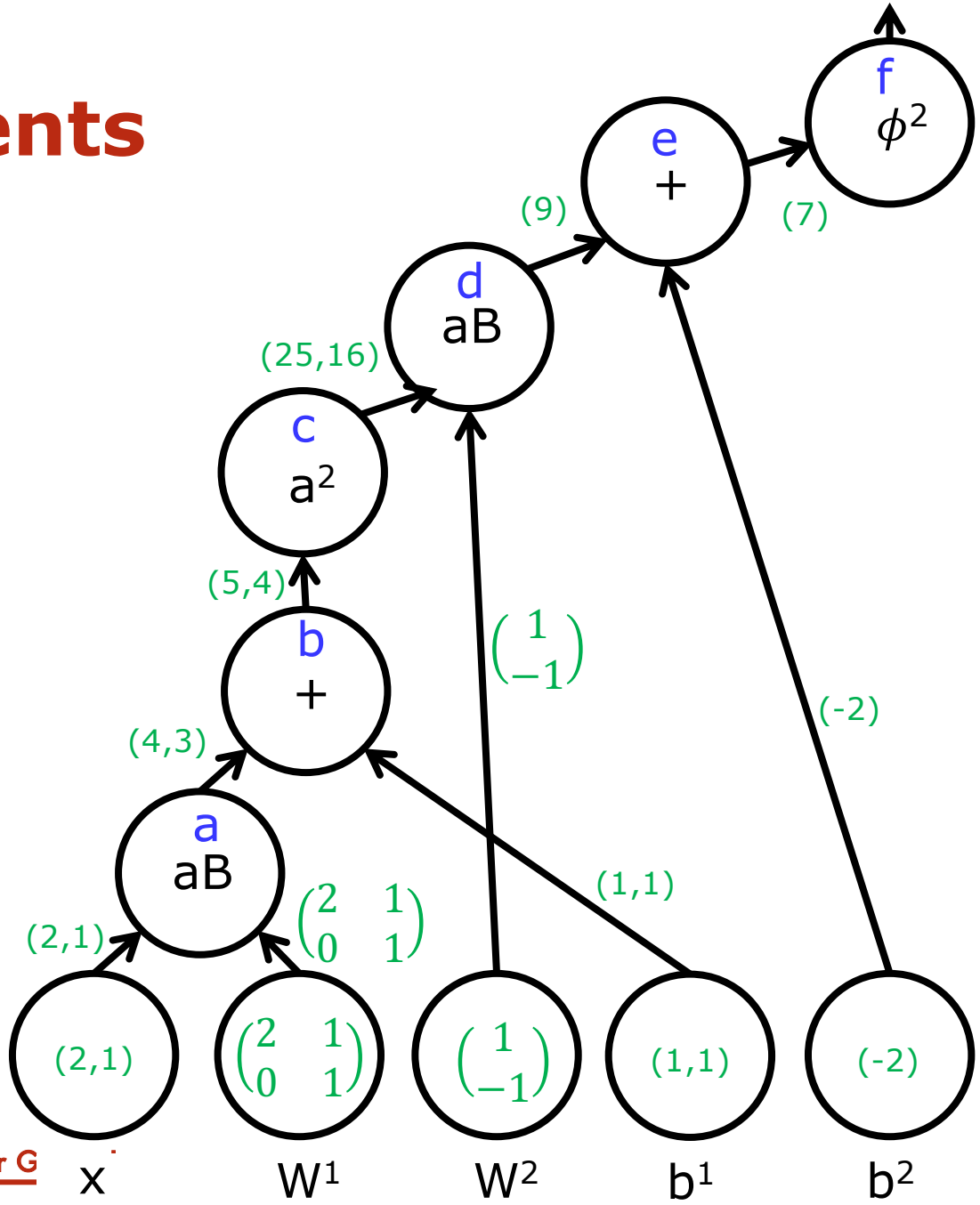
Computing Gradients

- We saw earlier:
  - Name nodes.
  - Forward pass.



Computing Gradients

- We saw earlier:
  - Name nodes.
  - Forward pass.



Computing Gradients

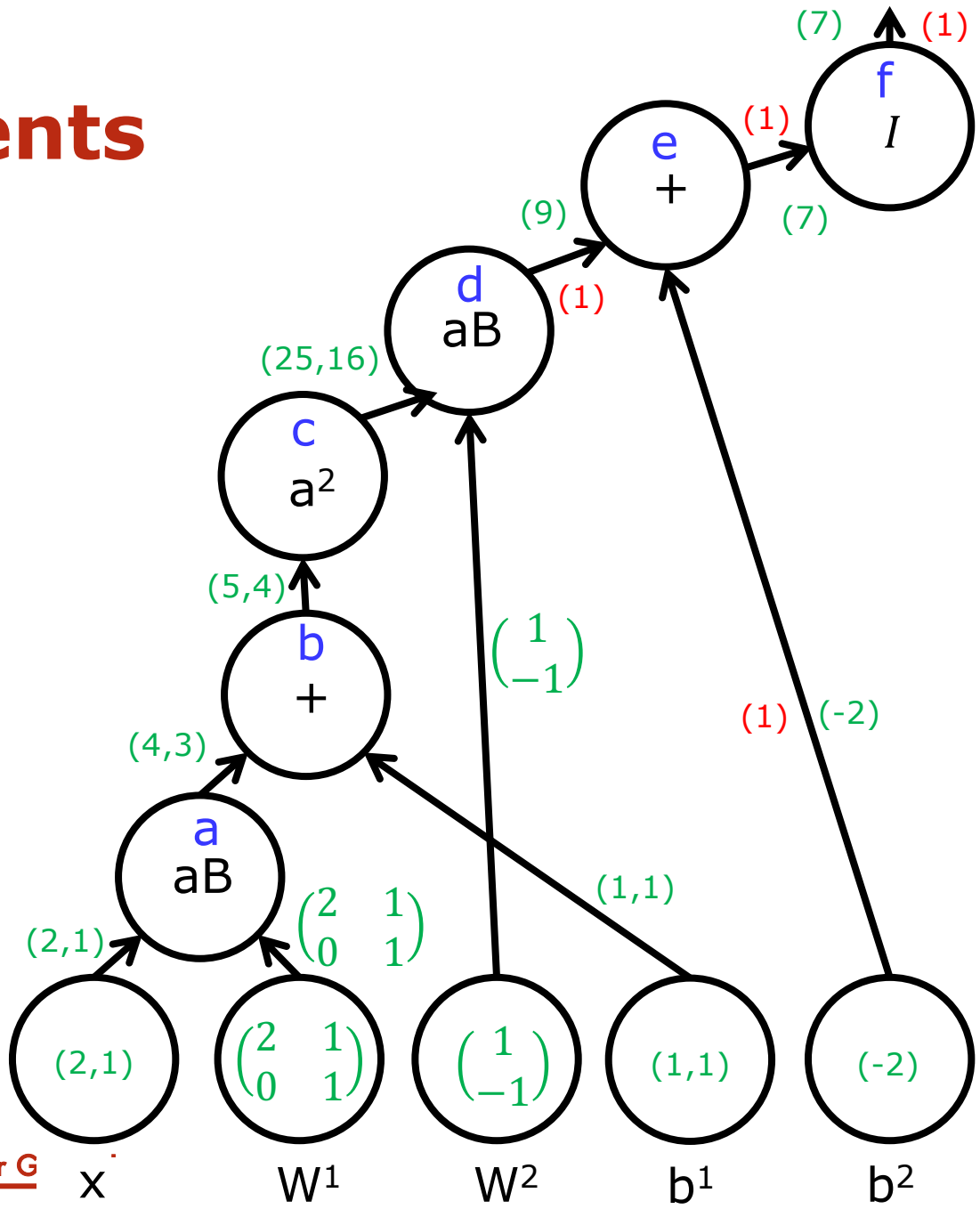
- We saw earlier:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial f} = 1$$

$$\frac{\partial f}{\partial e} = \frac{\partial e}{\partial e} = 1$$

$$\frac{\partial f}{\partial d} = \frac{\partial f}{\partial e}\frac{\partial e}{\partial d} = 1 * \frac{\partial d + b^2}{\partial d} = 1$$

$$\frac{\partial f}{\partial b^2} = \frac{\partial f}{\partial e}\frac{\partial e}{\partial b^2} = 1 * \frac{\partial d + b^2}{\partial b^2} = 1$$



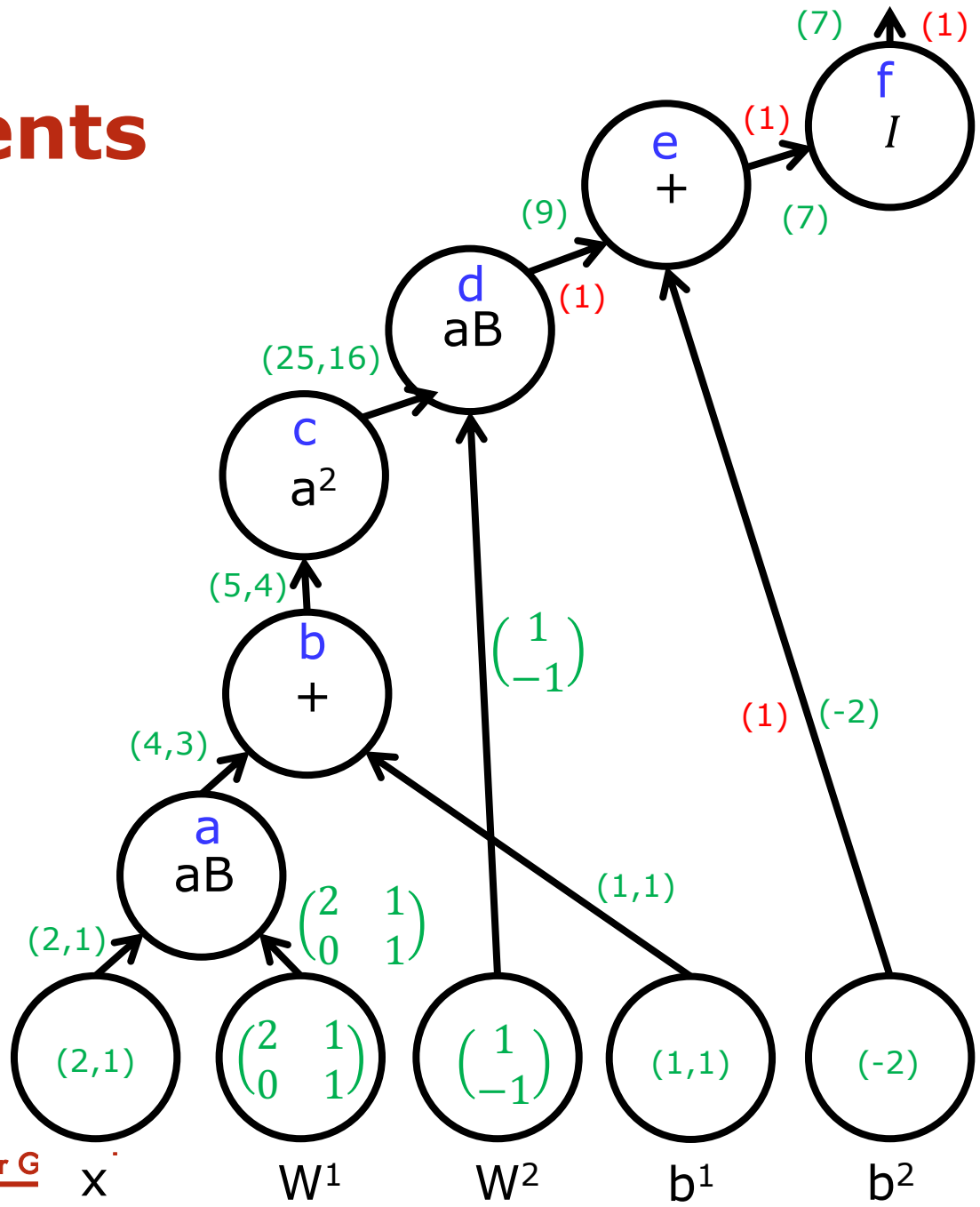
Computing Gradients

- We saw earlier:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial c} = \frac{\partial f}{\partial d}\frac{\partial d}{\partial c} = (1)\ \frac{\partial d}{\partial c}$$

c is not a scalar!

Expression still correct, only $\frac{\partial d}{\partial c}$ is the matrix of derivatives



Matrix of Derivatives

- Functions of multiple variables to multiple outputs:
  - If $l = (l_1, \ldots, l_k)$ is a function of variables $x_1,\ldots x_d$, i.e. from R^d to R^k.
  - Assume each l_i differentiable.

- Matrix of derivatives (a.k.a. Jacobian) is k x d:
  - $$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \cdots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \cdots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$$

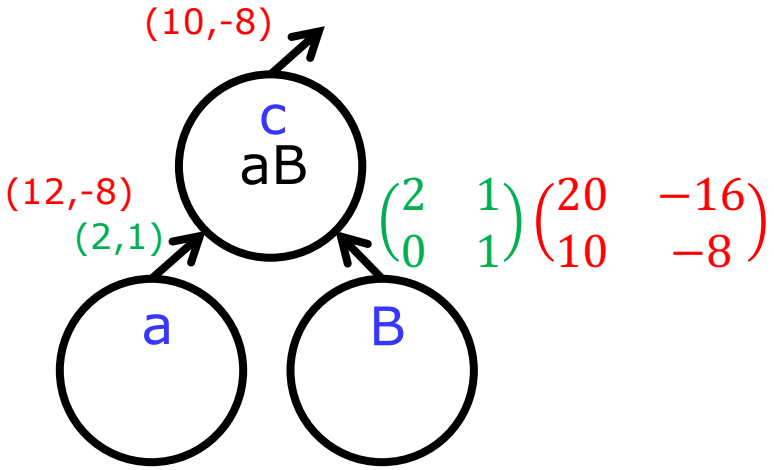
Computing Gradients

- We saw earlier:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

- Most tricky case aB.
  - Just memorize:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c} B^T$$

$$\frac{\partial f}{\partial B} = \begin{pmatrix} a_1 \frac{\partial f}{\partial c} \\ \vdots \\ a_k \frac{\partial f}{\partial c} \end{pmatrix} = a^T \frac{\partial f}{\partial c}$$



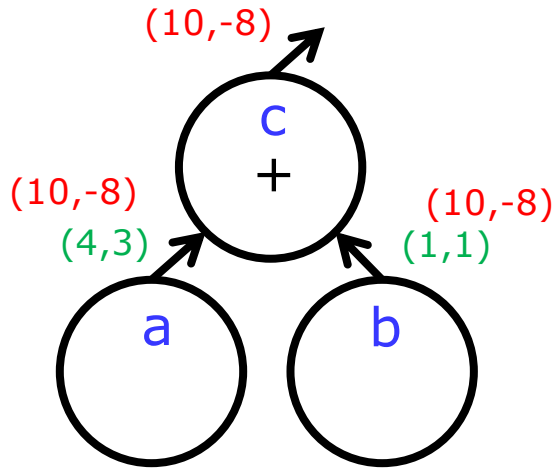
Computing Gradients

- We saw earlier:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

- Easy Case.
  - Just memorize:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c} I = \frac{\partial f}{\partial c}$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c} I = \frac{\partial f}{\partial c}$$

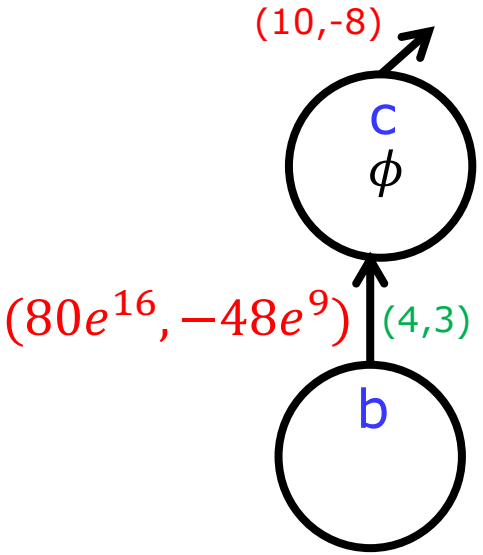


Computing Gradients

- We saw earlier:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

- Activations.
  - Just memorize:

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = \frac{\partial f}{\partial c}\begin{pmatrix} \phi'(b_1) & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \phi'(b_k) \end{pmatrix}$$



$$\phi(x) = e^{x^2}$$

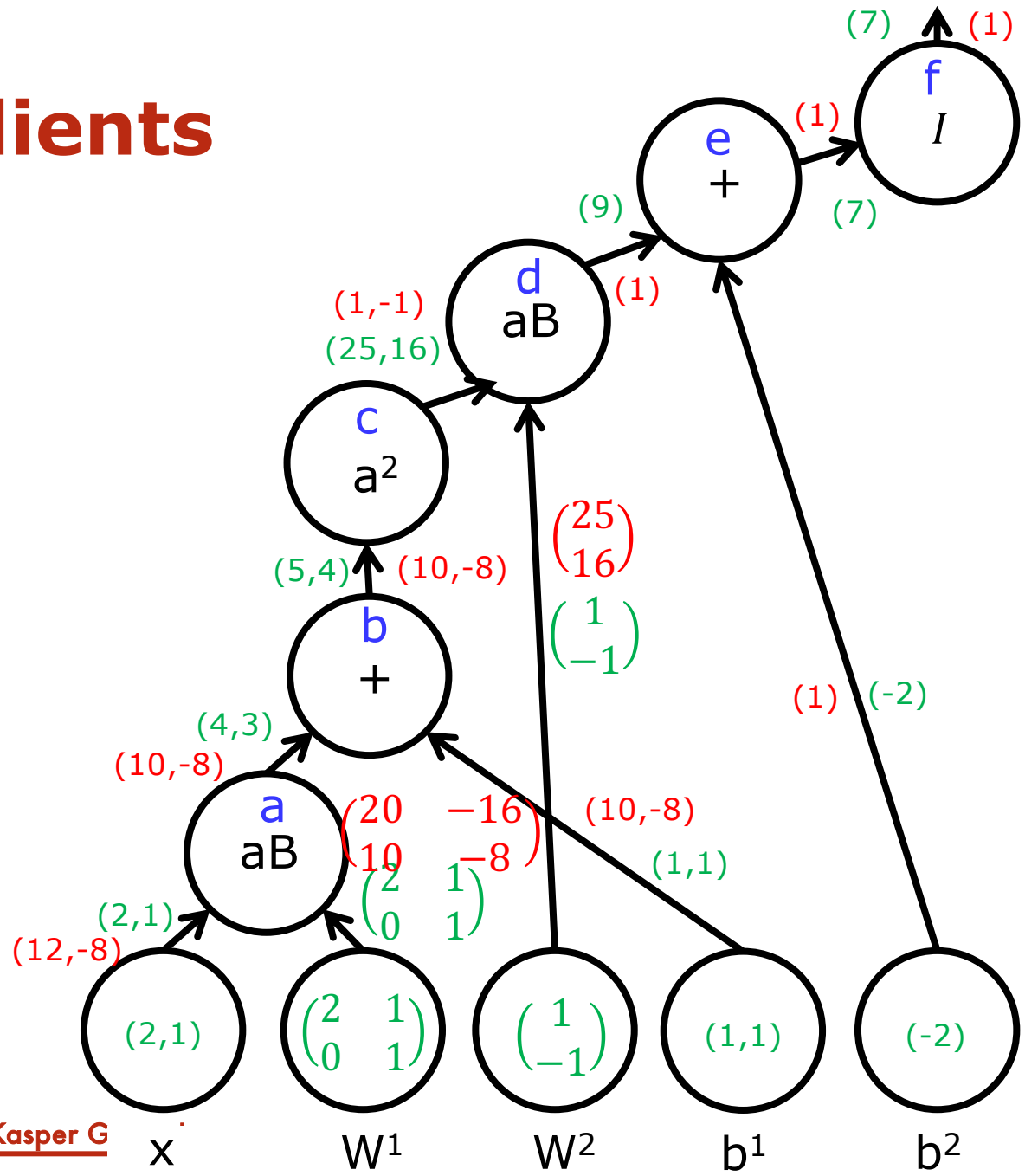
$$\phi'(x) = e^{x^2}2x$$

$$(10, -8)\begin{pmatrix} e^{16}8 & 0 \\ 0 & e^{9}6 \end{pmatrix}$$

Computing Gradients

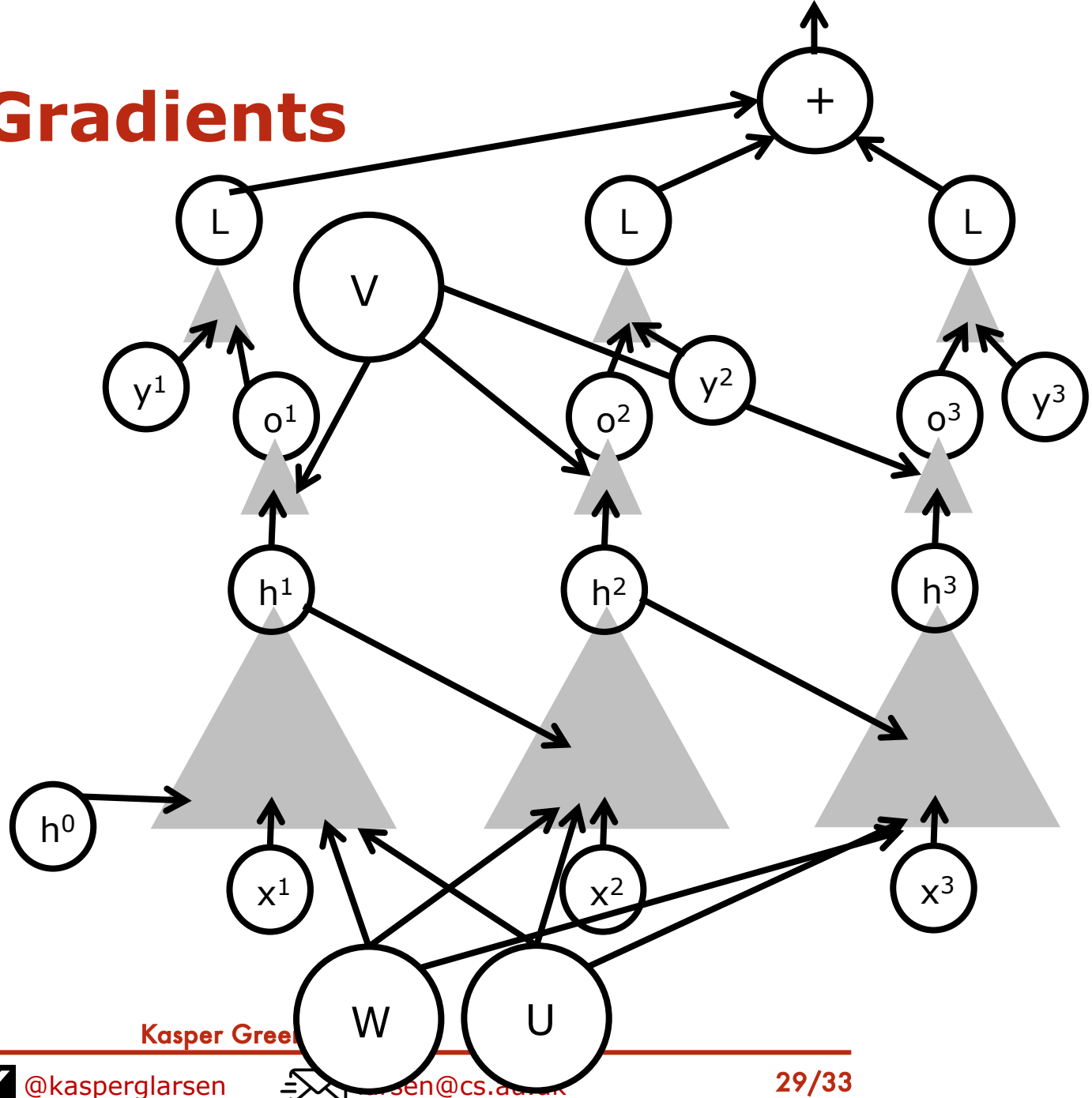
- We saw earlier:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

- Very tedious by hand.
  - But doable.

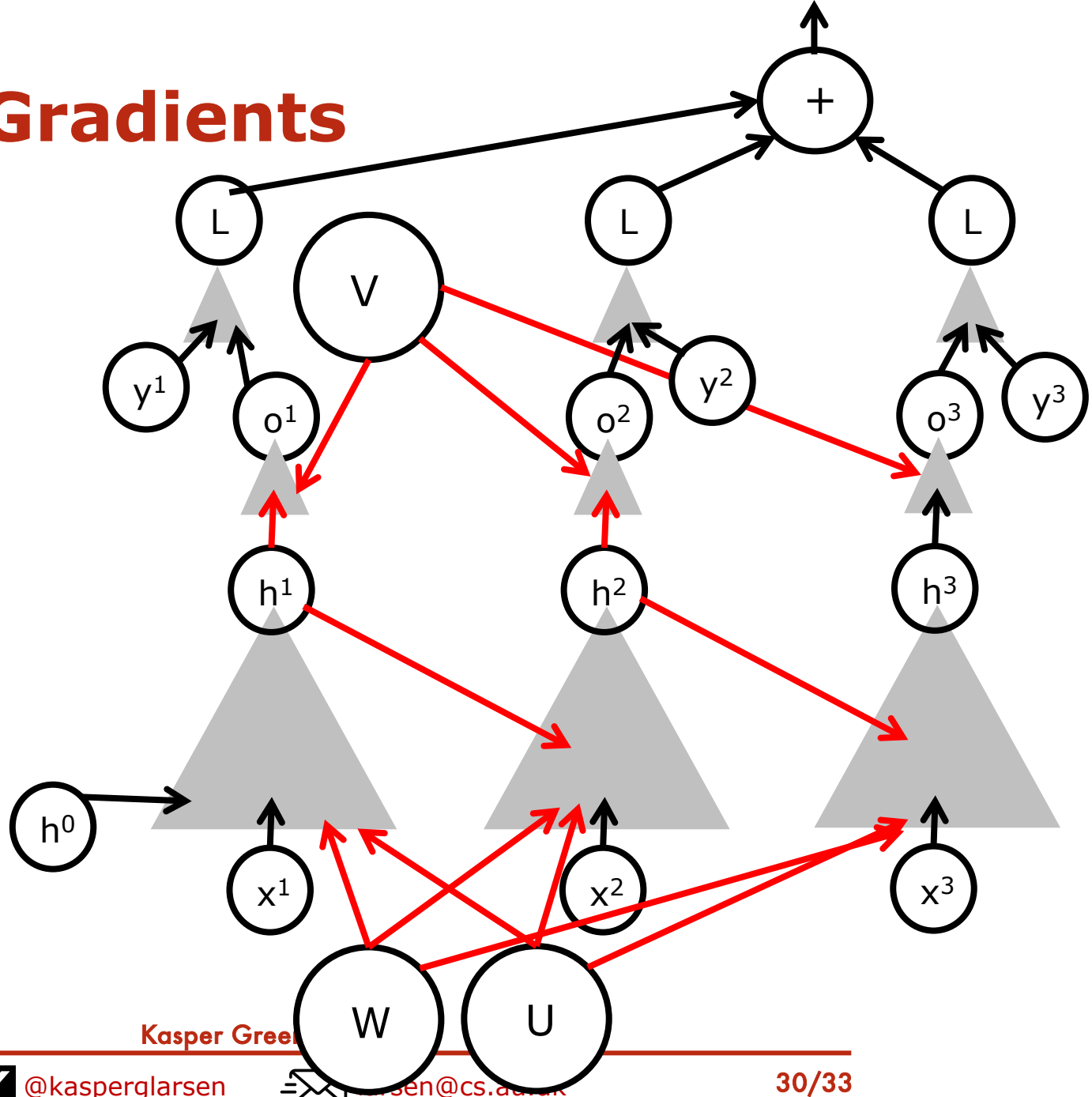


Computing Gradients

$$L(s) = \sum_{t=1}^{T} L(o^t, y^t)$$



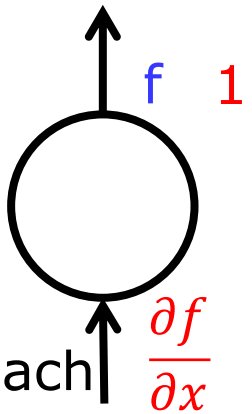
Computing Gradients

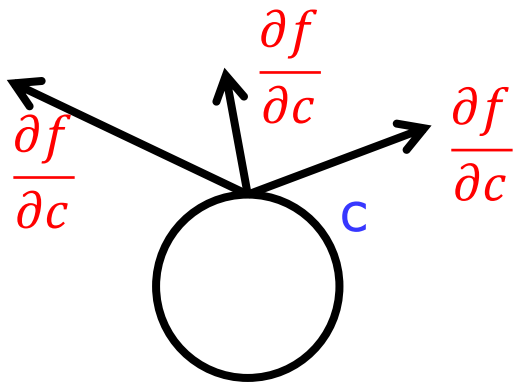


Variables re-used

Backpropagation

- Special Rule:
  - If a node sends output to multiple destinations:
    - Backprop computes a derivative on each edge.
    - These must be summed to obtain actual $\frac{\partial f}{\partial c}$ to be used further down the computation graph.

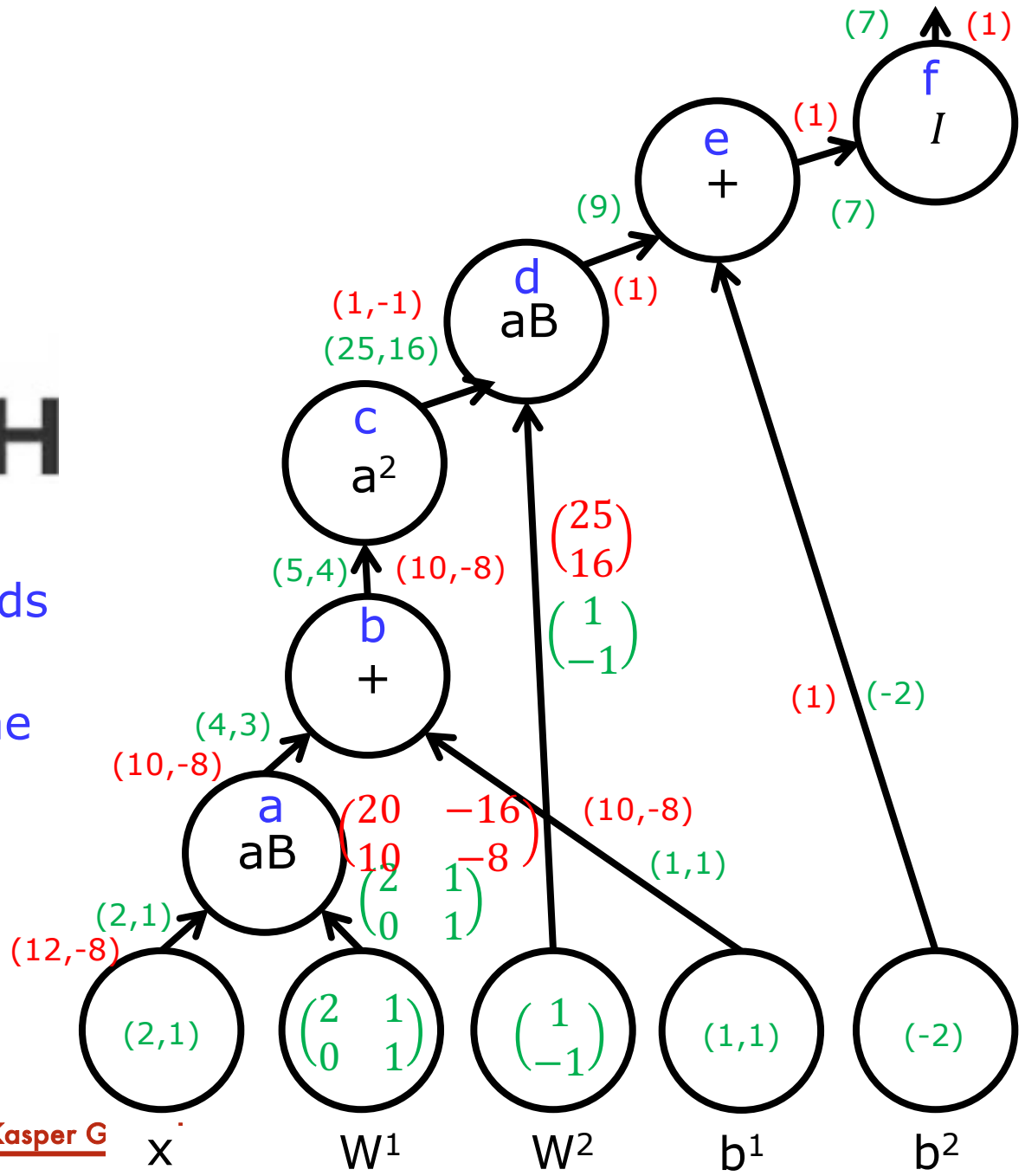




PyTorch



- Automates the backwards pass.
- Need only implement the forward pass.



@kasperglarsen

Stochastic Gradient Descent

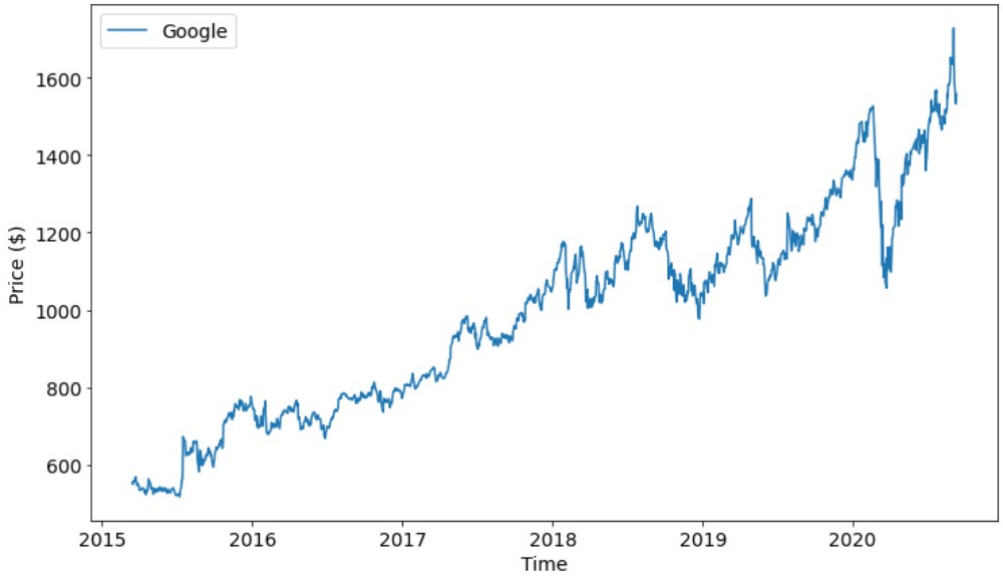
$o^t(x_i)$ output of recurrent neural net in step t on sequence $s_i=(x_i,y_i)$ with weights U,V,W

- Any point-wise loss:
  - $\mathrm{E}_{\mathrm{in}}(\mathrm{U},V,W) = \frac{1}{n}\sum_i \sum_{t=1}^{T} L(o^t(x_i), y_i^t)$
- Mini-batch SGD:
  - Initialize U,V,W (e.g. random N(0,1) distributed entries).
  - While more time:
    - Permute $(x_1,y_1),\ldots,(x_n,y_n)$ randomly.
    - For i=1,...,n/B:
      - Let $(x_1',y_1'),\ldots,(x_B',y_B')$ be next B data items.
      - $g = \frac{1}{B}\sum_{i=1}^{B} \sum_{t=1}^{T} \nabla_{U,V,W} L(o^t(x_i), y_i^t)$
      - U,V,W = U,V,W - ηg
  - Return w.

Recurrent Neural Nets Considerations

Machine Learning on Sequential Data

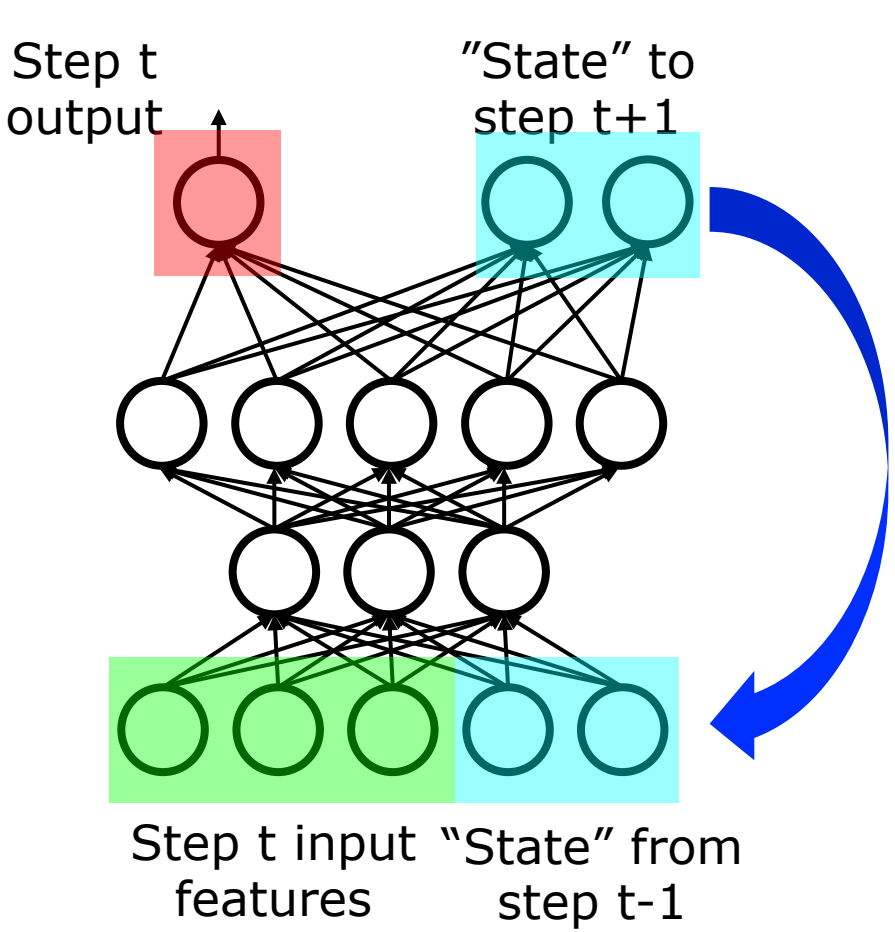
- Example Sequential Data:



- Stock Prices: Historic development of Google stock prices.
- Goal: Predict stock price tomorrow.

Recurrent Neural Nets

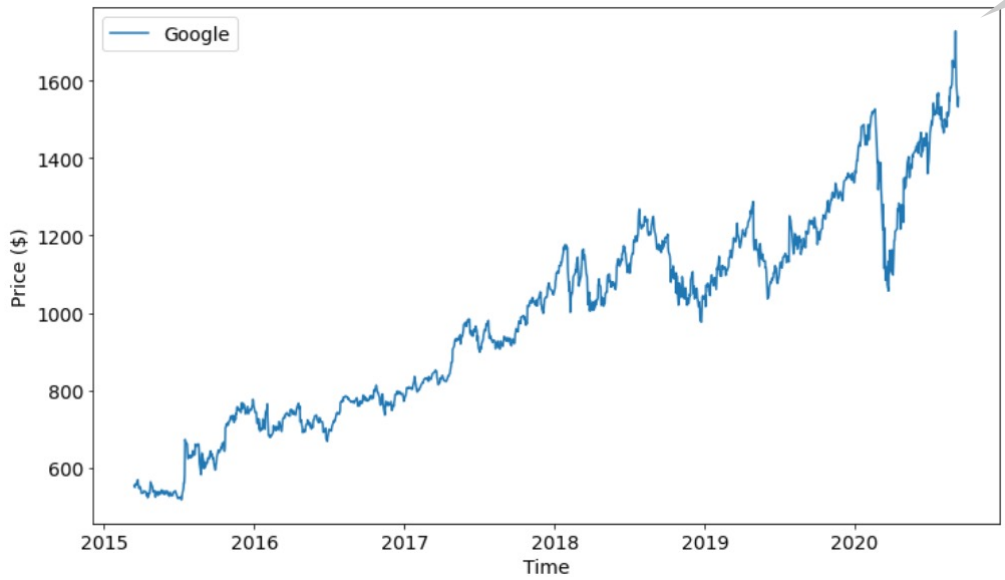
- Variant of Neural Nets for sequential data



Few But Long Sequences

We only have 1 training sequence, not n

- Example Sequential Data:



Features: $(x^1, x^2, \ldots, x^T)$

Labels: $(y^1, y^2, \ldots, y^T)$

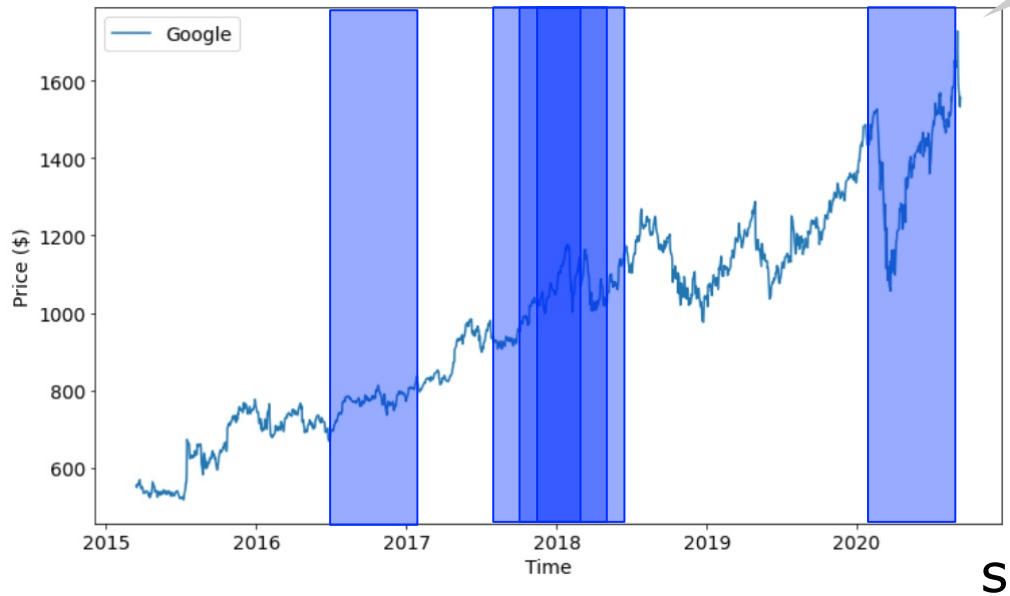
Google

- Stock Prices: Historic development of Google stock prices.
- Goal: Predict stock price tomorrow

Few But Long Sequences

We only have 1 training sequence, not n

- Example Sequential Data:



Features: $(x^1,x^2,\ldots,x^T)$

Labels: $(y^1,y^2,\ldots,y^T)$

Google

- Idea: Create short sequences from long
  - For fixed length, say 20, create every consecutive subsequence of length 20.

$s^1=((x^1,\ldots,x^{19}),(y^1,\ldots,y^{19}))$

$s^2=((x^2,\ldots,x^{20}),(y^2,\ldots,y^{20}))$

$s^3=((x^3,\ldots,x^{21}),(y^3,\ldots,y^{21}))$

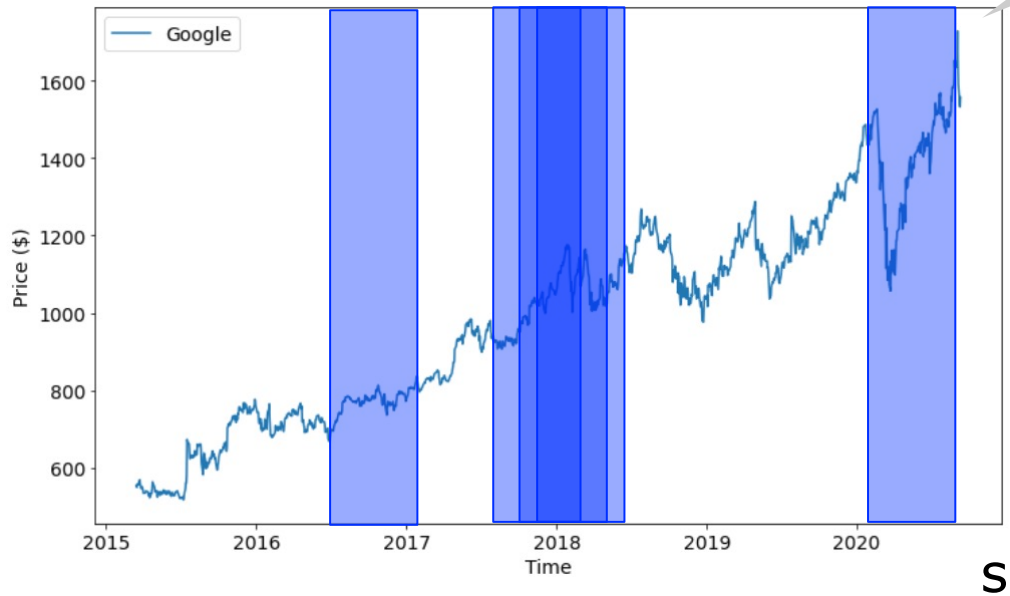
$s^4=((x^4,\ldots,x^{22}),(y^4,\ldots,y^{22}))$

...

Few But Long Sequences

We only have 1 training sequence, not n

- Example Sequential Data:



Features: $(x^1,x^2,\ldots,x^T)$

Labels: $(y^1,y^2,\ldots,y^T)$

Google

- New Predictions:
  - Feed in past 20 steps.
  - Could also work well with more or less steps

$s^1=((x^1,\ldots,x^{19}),(y^1,\ldots,y^{19}))$

$s^2=((x^2,\ldots,x^{20}),(y^2,\ldots,y^{20}))$

$s^3=((x^3,\ldots,x^{21}),(y^3,\ldots,y^{21}))$

$s^4=((x^4,\ldots,x^{22}),(y^4,\ldots,y^{22}))$

...

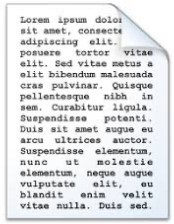
Only One Label

- Example Sequential Data:

Features: $(x^1, x^2, \ldots, x^T)$

Each word a feature

E.g. one-hot or word-embedding



"We shall execute the transaction under a legitimate…"

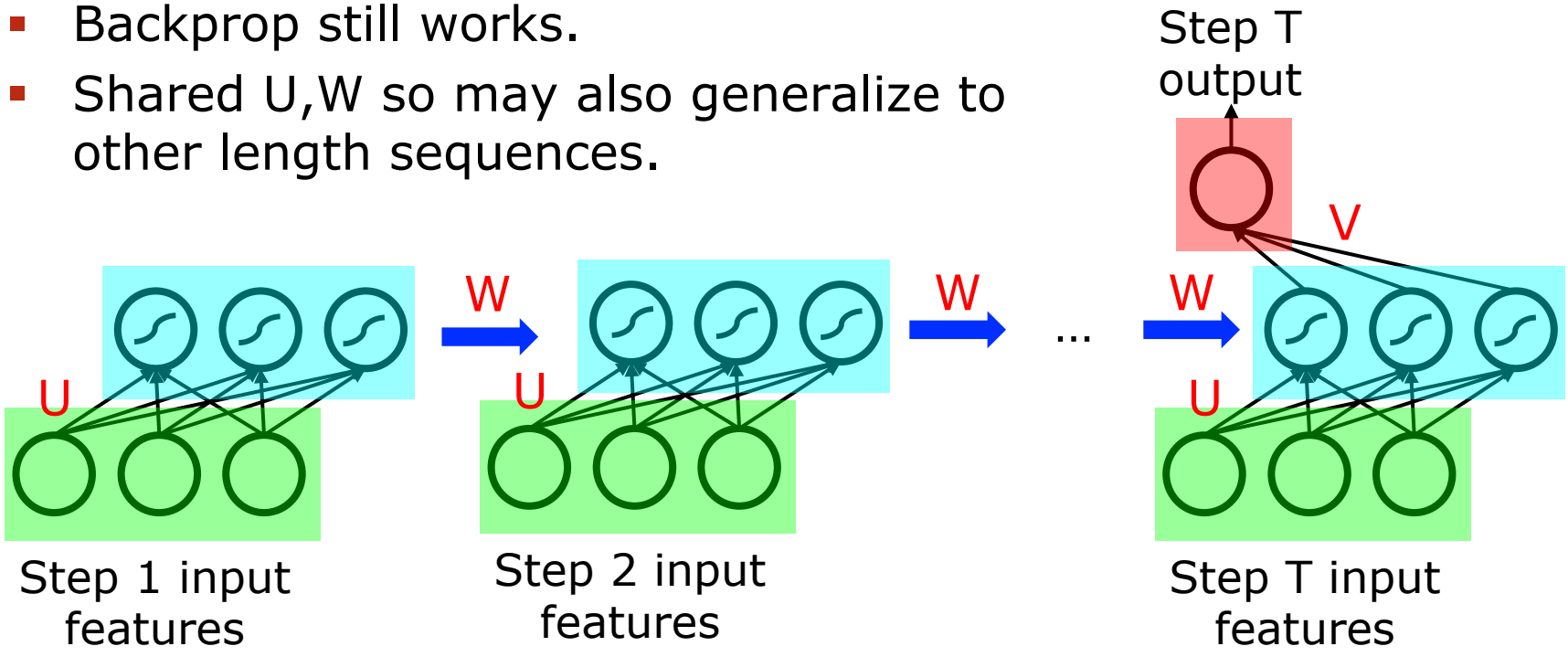
Labels: ??

Only one label on whole sequence: spam/not-spam

- Text: Variable length sentences
- Goal: Spam or not spam?

Only One Label

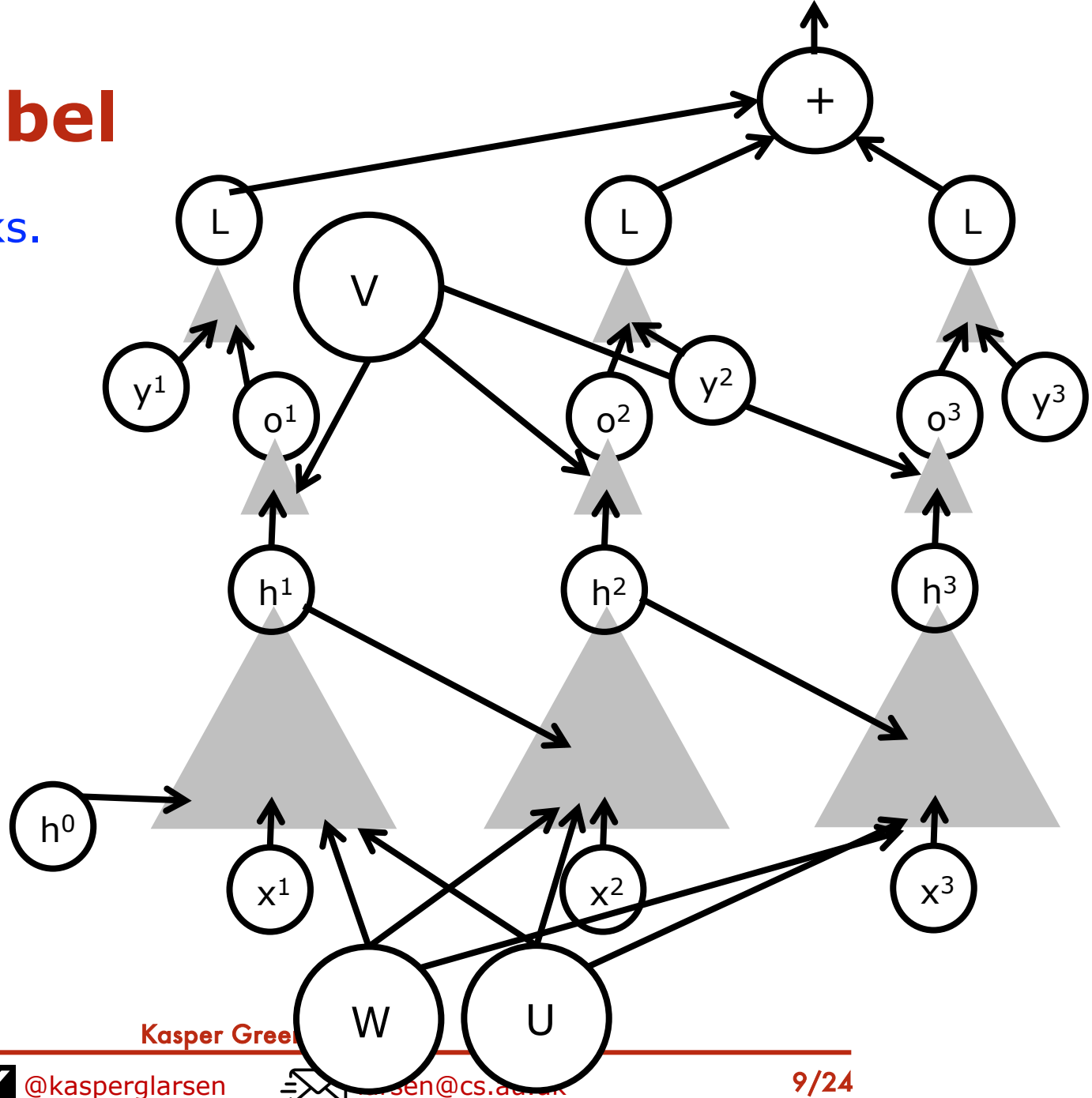
- Only one label for whole sequence
  - Just run forward pass without computing an output each time.
  - Backprop still works.
  - Shared U,W so may also generalize to other length sequences.



Only One Label

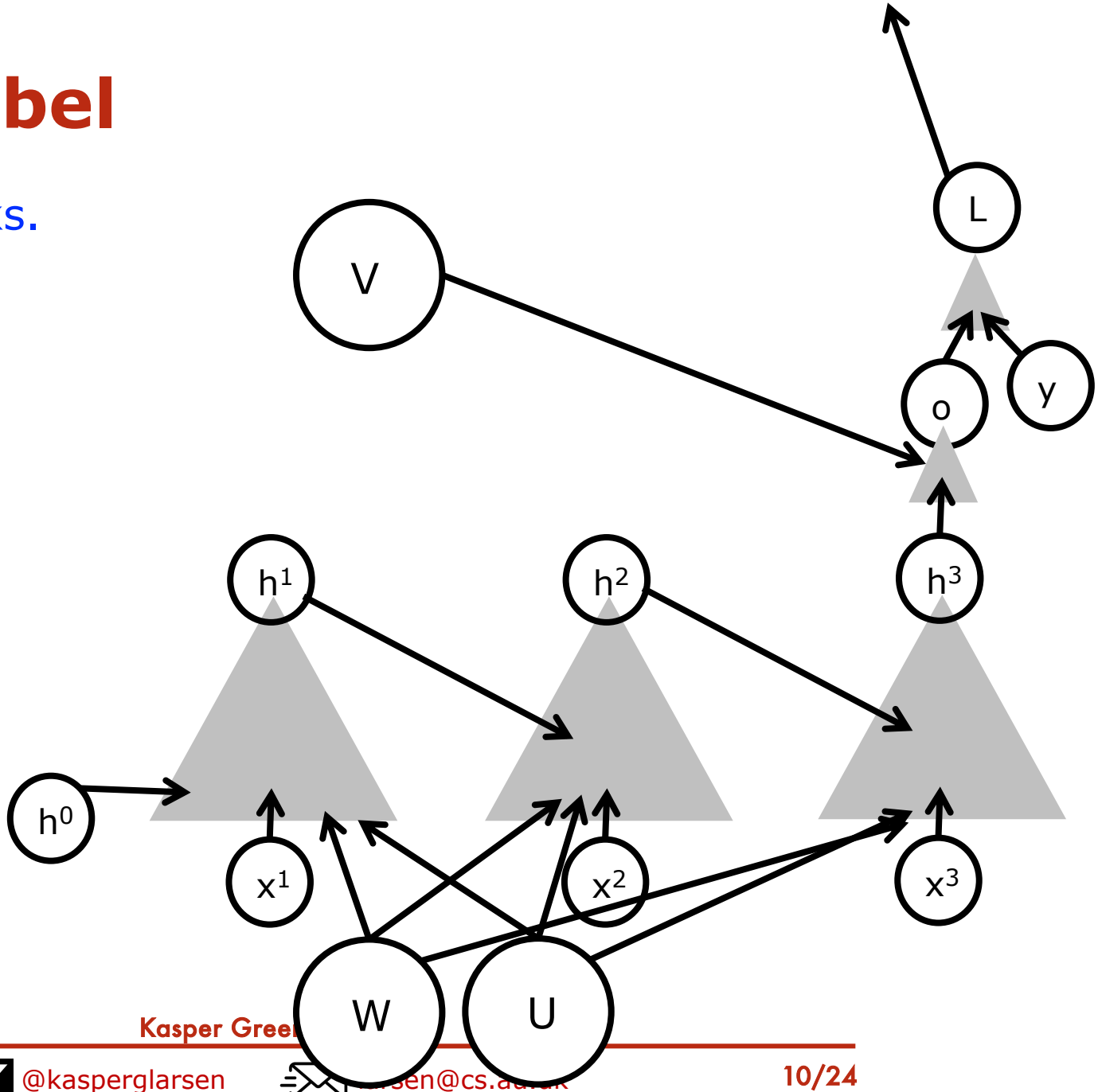
- Backprop still works.

$$L(s) = \sum_{t=1}^{T} L(o^t, y^t)$$



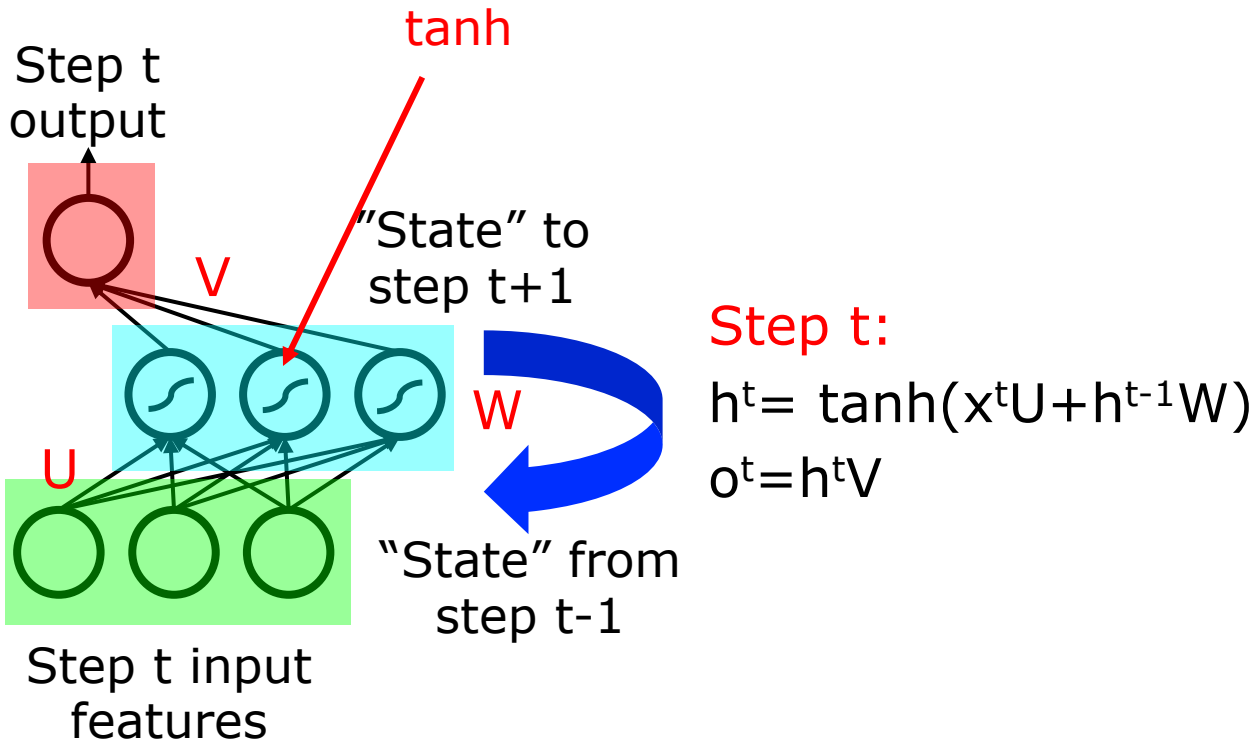
Only One Label

- Backprop still works.



Activation Functions

- We insisted on hyperbolic tangent activation

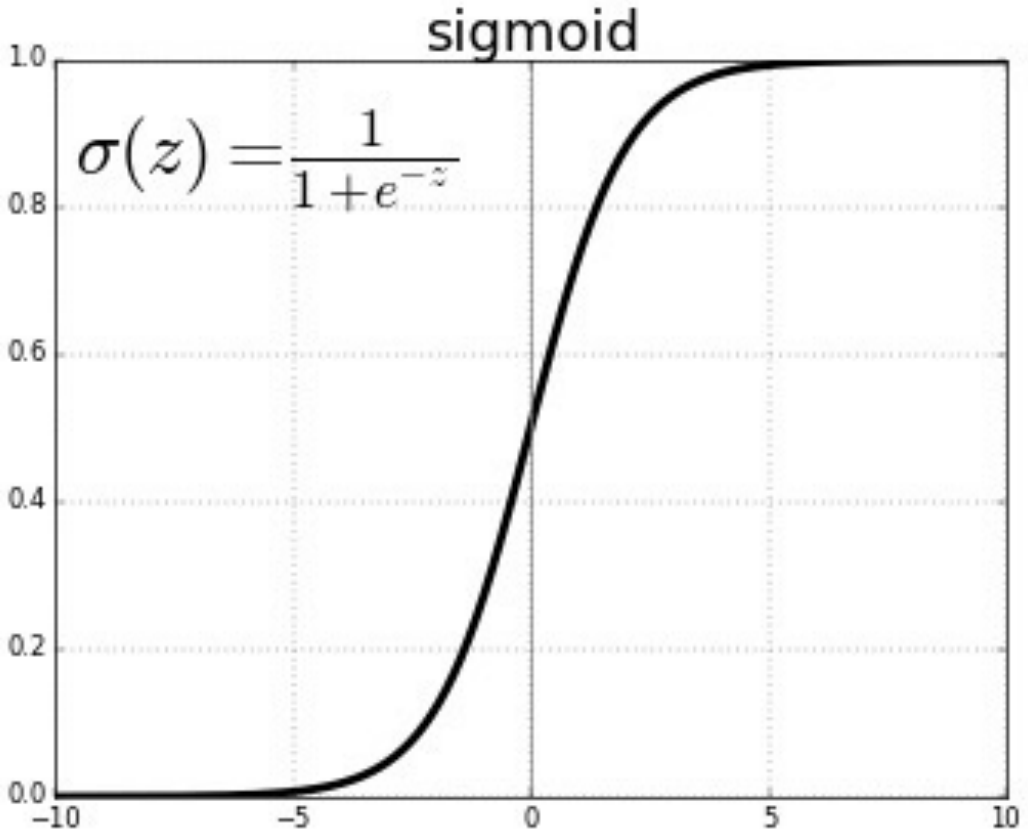


- Features: $(x^1, x^2, \ldots, x^t, \ldots)$

Sigmoid

- Logistic Sigmoid

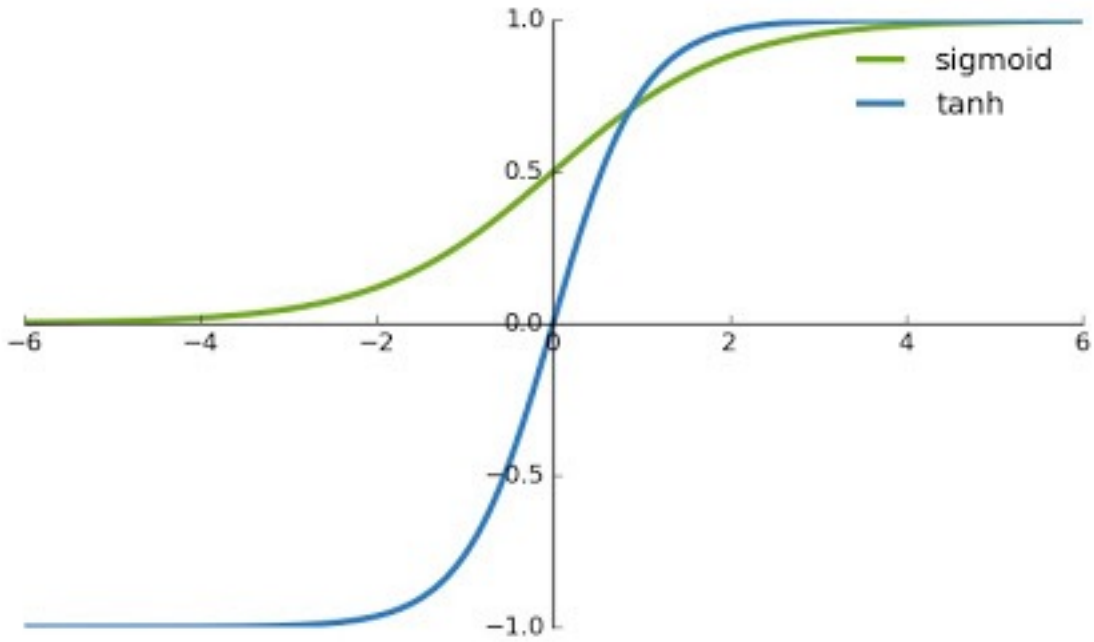
- Sigmoid $\sigma(z)$
  - ”Fires” as z becomes big enough.



Hyperbolic Tangent

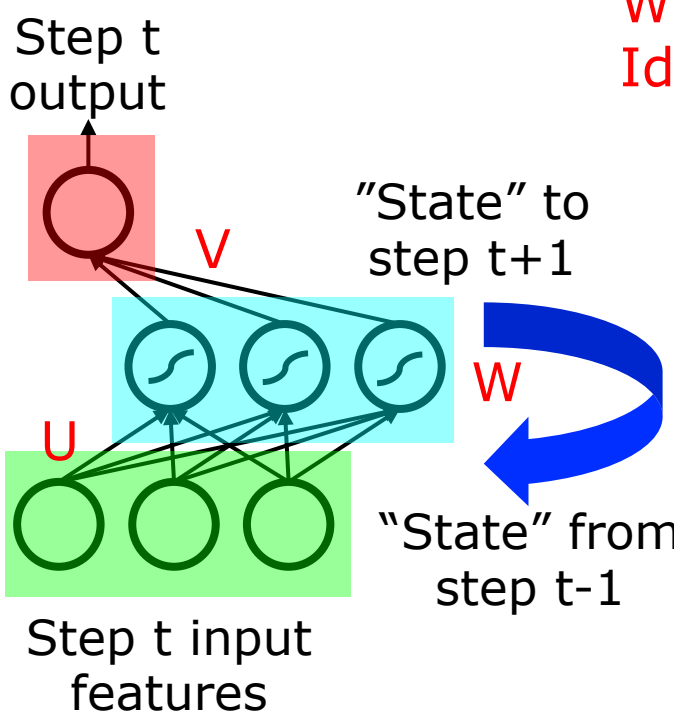
- Hyperbolic Tangent

- $\tanh(z)$
  - $\tanh(z) = 2\sigma(2z) - 1$



Activation Functions

- We insisted on hyperbolic tangent activation



What happens with Identity activation?

Step t:

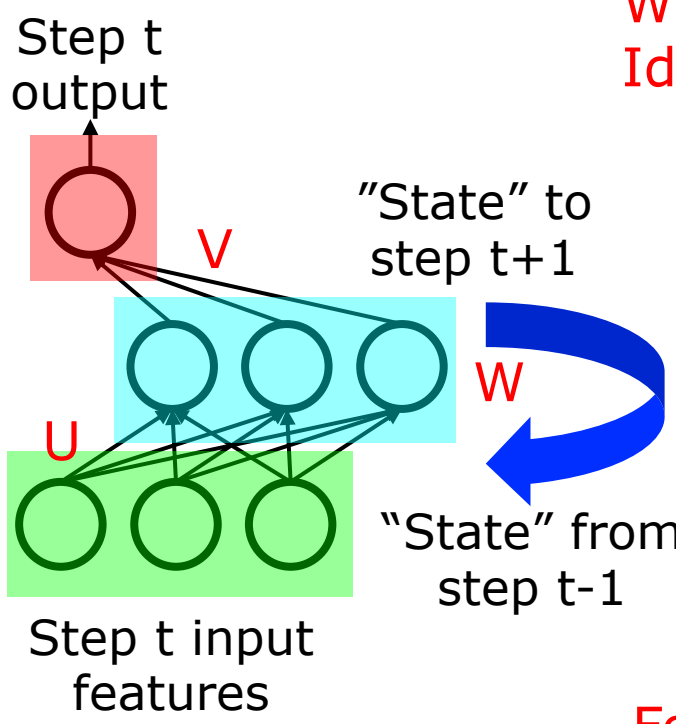
$h^t = \tanh(x^tU + h^{t-1}W)$

$o^t = h^tV$

- Features: $(x^1, x^2, \ldots, x^t, \ldots)$

Activation Functions

- We insisted on hyperbolic tangent activation



What happens with Identity activation?

Step t:

$h^t = x^t U + h^{t-1} W$

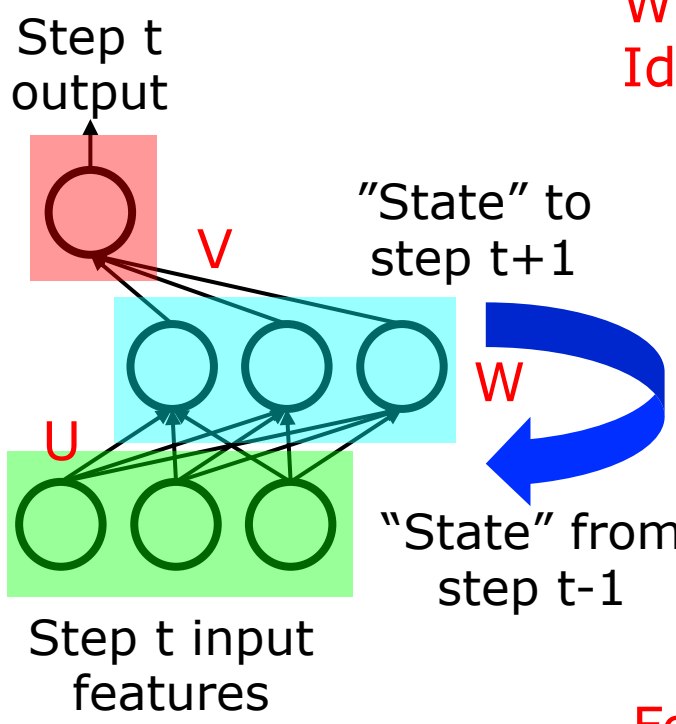
$o^t = h^t V$

For clarity of example, let us assume x^t for $t>1$ is 0 and $h^0=0$.

- Features: $(x^1, x^2, \ldots, x^t, \ldots)$

Activation Functions

- We insisted on hyperbolic tangent activation



What happens with Identity activation?

Step t:

$h^t = h^{t-1}W$

$h^t = (x^1U)W\ldots W$

$h^t = (x^1U)W^t$

- Features: $(x^1, x^2, \ldots, x^t, \ldots)$

For clarity of example, let us assume x^t for $t>1$ is 0 and $h^0=0$.

Activation Functions

- Exploding or Vanishing Values
  - For clarity: $x^t=0$ for $t>1$ and $h^0=0$.

  - $h^t= (x^1U)W^t$
  - Let $z=x^1U$. Then $h^t= zW^t$

- Similar to so-called Power Method.
- For simplicity, let us assume that W is symmetric.
- Eigendecomposition:
  - $W=EDE^T$
  - E orthonormal and D diagonal with singular values on diagonal.
  - $W^2=(EDE^T)(EDE^T)=ED^2E^T$
  - $W^t=ED^tE^T$

Eigenvalues raised to t'th power

Activation Functions

- Exploding or Vanishing Values
  - For clarity: $x^t=0$ for $t>1$ and $h^0=0$.

  - $h^t = (x^1U)W^t$
  - Let $z=x^1U$. Then $h^t = zED^tE^T$

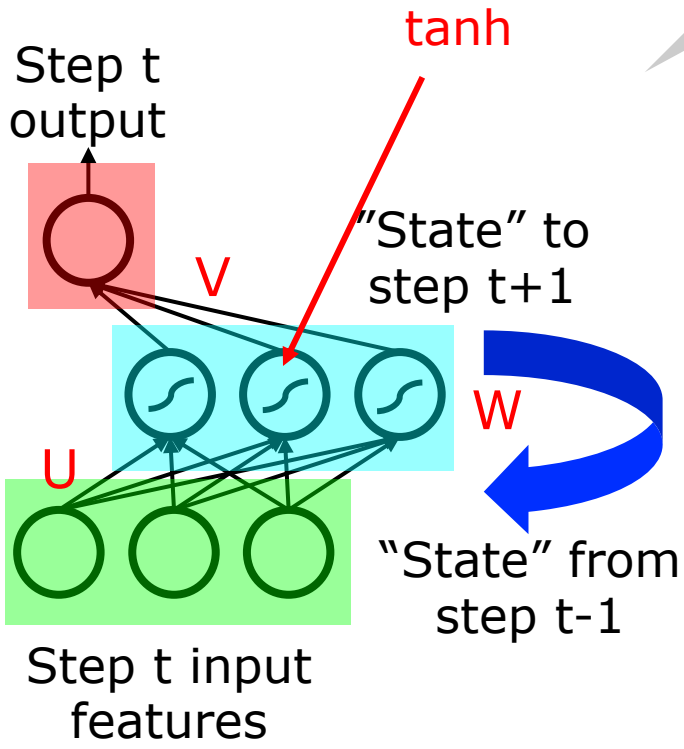
- Eigenvalue λ_i becomes λ_i^t. Let μ_i be corresponding eigenvector.
- Can be shown that $h^t = \sum_{i=1}^{n} \langle z, \mu_i \rangle \lambda_i^t \mu_i$

- If $\lambda_i > 1$ then $\|h^t\|$ may grow exponentially with number of steps!

- If all $\lambda_i < 1$ then $\|h^t\|$ shrinks exponentially with number of steps. Roughly means forget early steps!

Activation Functions

- We insisted on hyperbolic tangent activation



"Squeezes" values back to [-1,1] range

Avoids the exploding values

Step t:

$h^t = \tanh(x^tU + h^{t-1}W)$

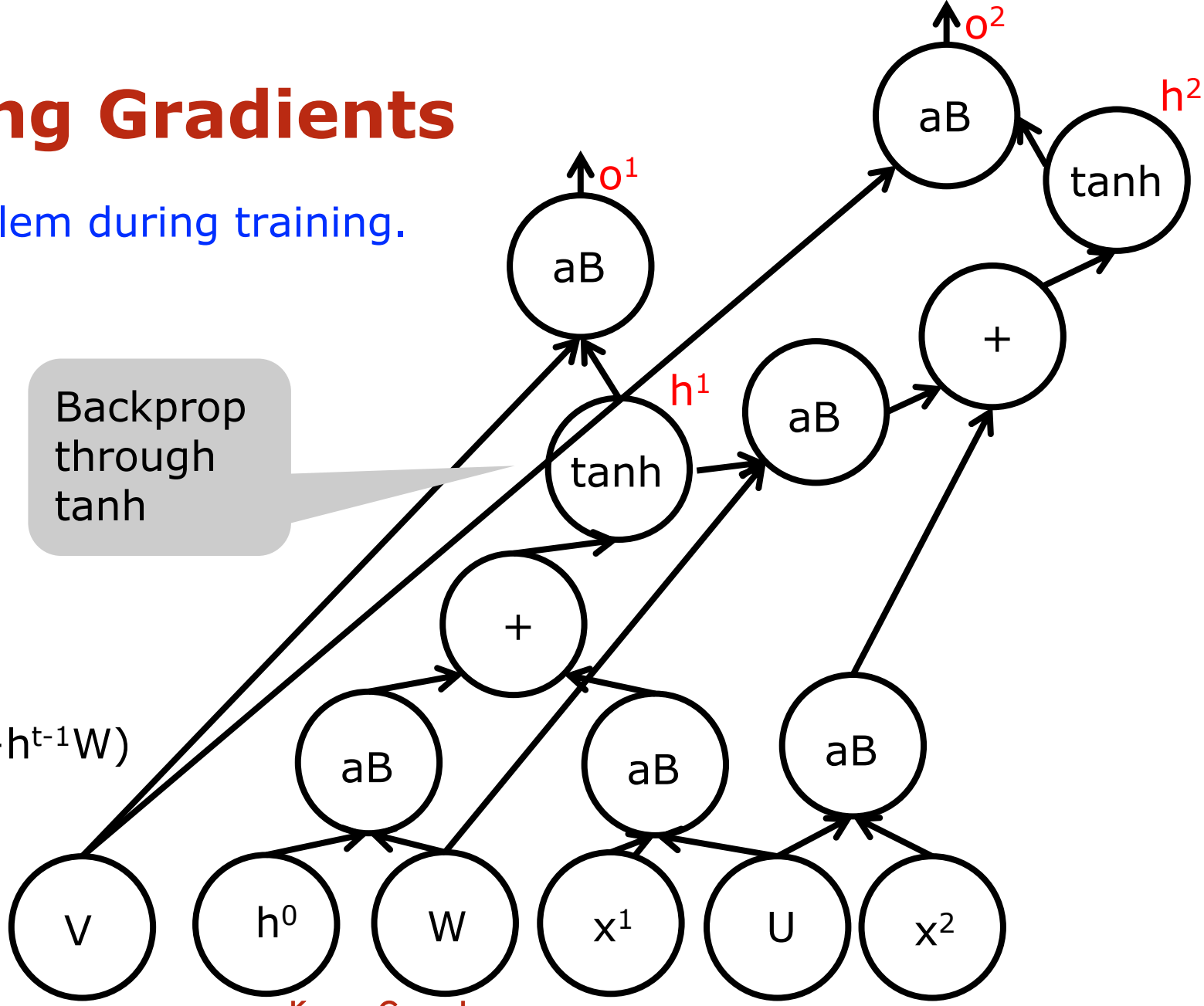
$o^t = h^tV$

"Forgetting" very early steps is a well-known problem for RNNs.

- Features: $(x^1, x^2, \ldots, x^t, \ldots)$

Vanishing Gradients

- Also a problem during training.



Step t:

$h^t = \tanh(x^tU + h^{t-1}W)$

$o^t = h^tV$

Vanishing Gradients

- We saw earlier:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

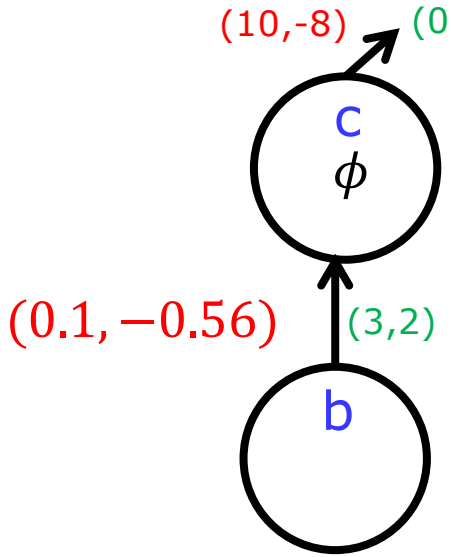
- Activations.
  - Just memorize:

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} =$$

$$\frac{\partial f}{\partial c}\begin{pmatrix} \phi'(b_1) & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \phi'(b_k) \end{pmatrix}$$

$$\tanh(3) \approx 0.995$$
$$1 - \tanh^2(3) \approx 0.01$$

$$\tanh(-2) \approx -0.964$$
$$1 - \tanh^2(-2) \approx 0.07$$

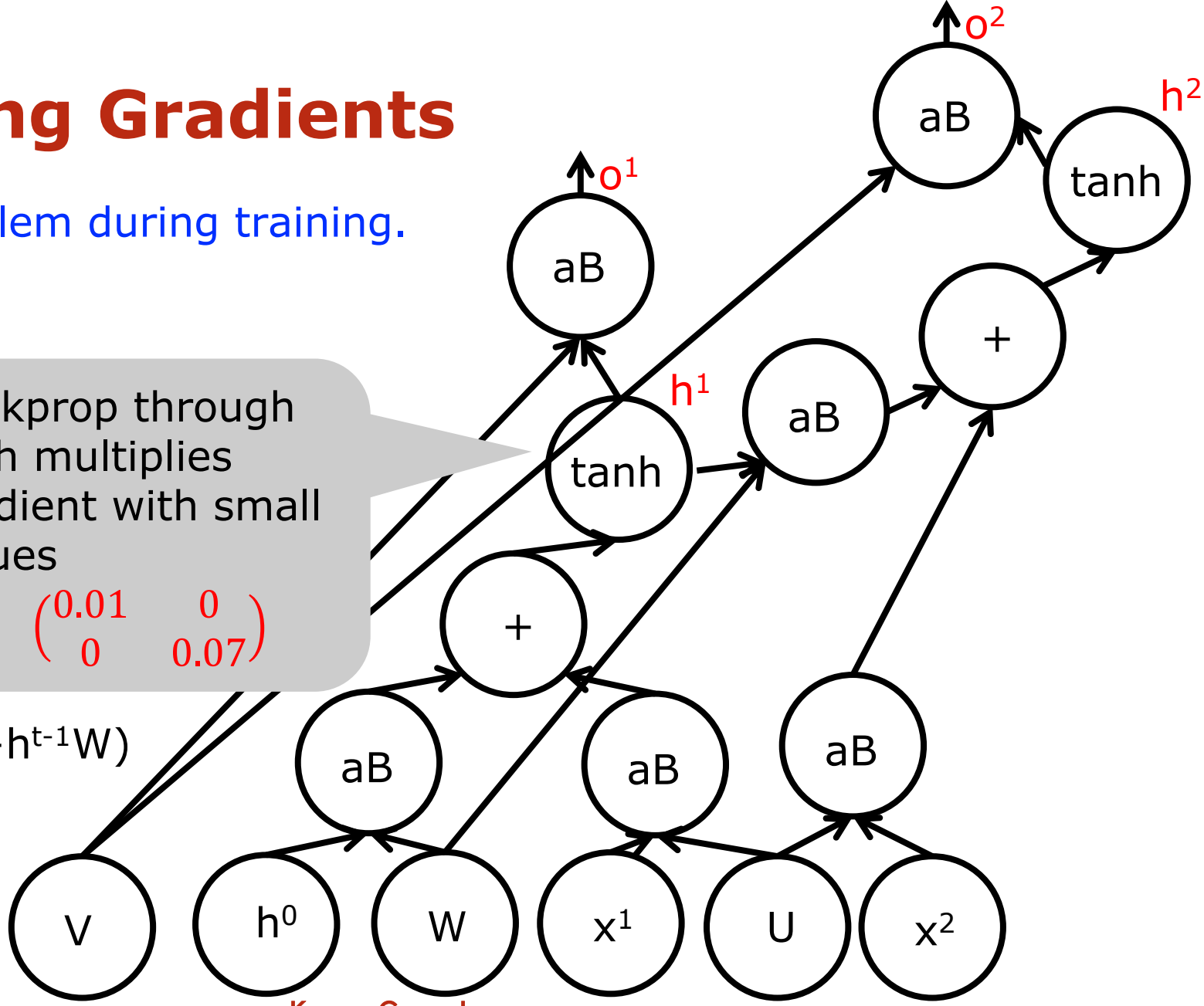


$$\phi(x) = \tanh(x)$$
$$\phi'(x) = 1 - \tanh^2(x)$$

$$(10, -8)\begin{pmatrix} 0.01 & 0 \\ 0 & 0.07 \end{pmatrix}$$

Vanishing Gradients

- Also a problem during training.



Backprop through tanh multiplies gradient with small values

$$\begin{pmatrix} 0.01 & 0 \\ 0 & 0.07 \end{pmatrix}$$

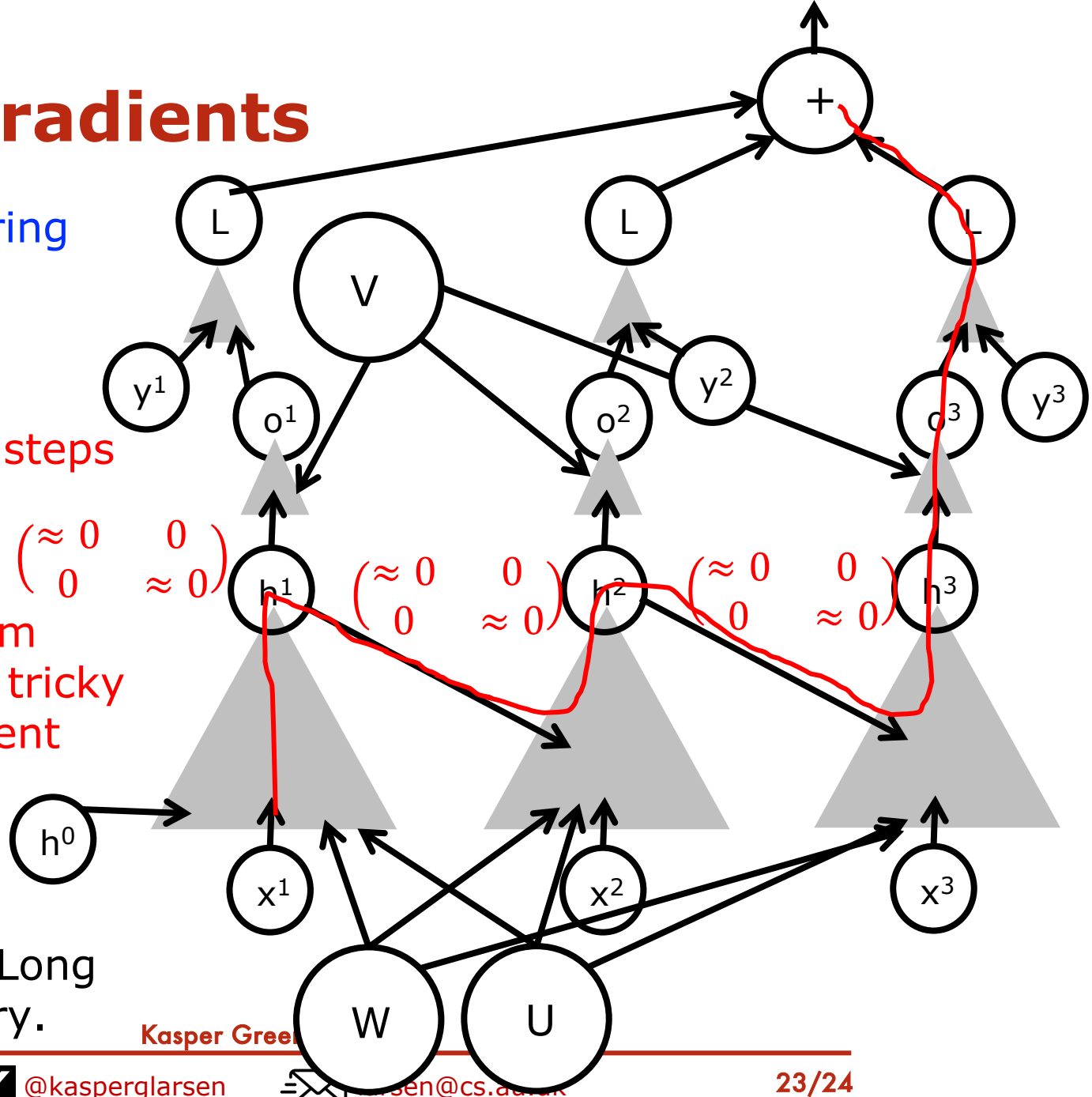
Step t:

$h^t = \tanh(x^tU + h^{t-1}W)$

$o^t = h^tV$

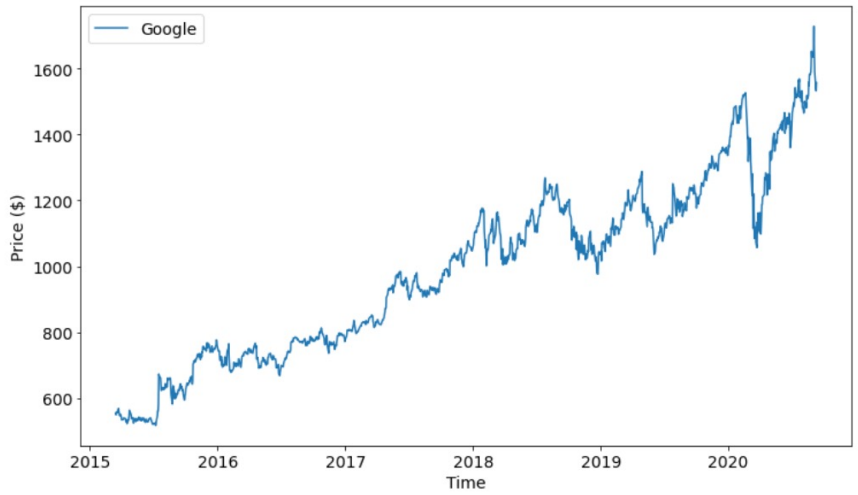
Vanishing Gradients

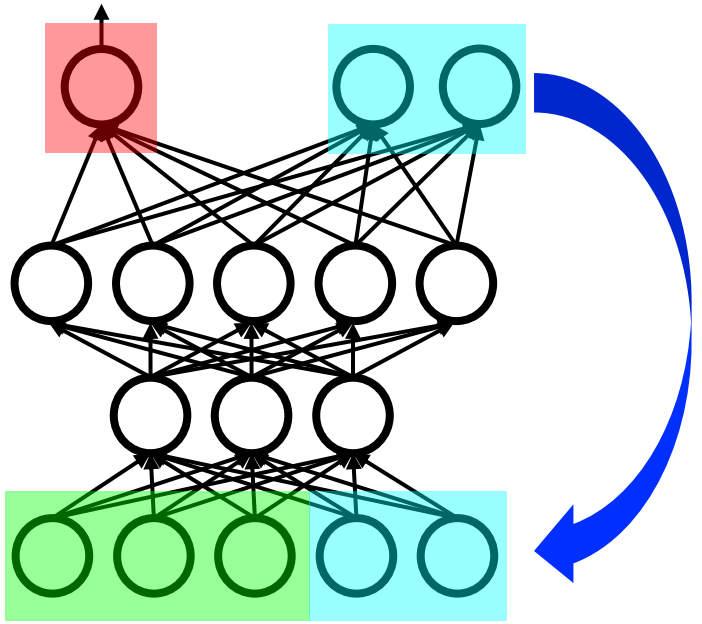
- Also a problem during training.
- Gradients go down exponentially with steps
- So-called ”long term dependencies” are tricky to learn for Recurrent Neural Nets
- Alternative LSTM: Long Short-Term Memory.



Summary

- Recurrent Neural Nets for learning sequential data





- Benefits:
  - Parameter sharing across time steps.
  - Variable length sequences.
  - Trained via Stochastic Gradient Descent and backprop.
- Drawback: Difficult to learn long-term dependencies.